

Privacy-Preserving Intrusion Detection Using Federated Learning: A Comparative Study of Centralized and Federated Deep Learning Models on NSL-KDD

*Report submitted to the SASTRA Deemed to be University
as the requirement for the course*

CSE300 - MINI PROJECT

Submitted by

ACHAKALA VENKATA SAI LIKITHA

(Reg No: 227003004, B.Tech.-CSE)

AISHWARYA K

(Reg No: 227003007, B.Tech.-CSE)

DIPNITHA S

(Reg No: 227003044, B.Tech.-CSE)

May 2026



Department of Computer Science and Engineering

SRINIVASA RAMANUJAN CENTRE
KUMBAKONAM, TAMIL NADU INDIA- 612001



SRINIVASA RAMANUJAN CENTRE
KUMBAKONAM - 612001

Bonafide Certificate

This is to certify that the report titled “Privacy-Preserving Intrusion Detection Using Federated Learning: A Comparative Study of Centralized and Federated Deep Learning Models on NSL-KDD” submitted as a requirement for the course, CSE300 - **MINI PROJECT** for B.Tech. is a bonafide record of the work done by **Ms. ACHAKALA VENKATA SAI LIKITHA (227003004, B.Tech CSE), Ms. AISHWARYA K (227003007, B.Tech CSE), Ms. DIPNITHA S (227003044, B.Tech CSE)** during the academic year 2025-26, in the Srinivasa Ramanujan Centre , under my supervision.

Signature of Project Supervisor

: 

Name with Affiliation

: **Dr. KALACHELVI V ACP / CSE / SRC /**

SASTRA Date

: **06-05-2026**

Mini Project Viva voce held on 06-05-2026

Examiner 1

Examiner 2

ACKNOWLEDGEMENTS

We would like to express our sincere gratitude to our Honorable Chancellor **Prof. R. Sethuraman**, for providing us with the opportunity and the necessary infrastructure to carry out this project as part of our curriculum.

We would like to thank our Honorable Vice-Chancellor, **Dr. S. Vaidhyasubramaniam**, and **Dr. S. Swaminathan**, Dean, Planning & Development, for their constant encouragement and strategic support throughout our college journey. We also extend our sincere thanks to **Dr. R. Chandramouli**, Registrar, SASTRA Deemed to be University, for providing us with the opportunity to pursue this project.

We extend our heartfelt thanks go to **Dr. B. Santhi**, Dean, **Dr. A. Alli Rani**, Associate Dean, and **Dr. D. Narasimhan**, Associate Dean, Srinivasa Ramanujan Centre, SASTRA Deemed to be University. We exhibit our pleasure in expressing our thanks to **Dr. V. Kalaichelvi**, Associate Professor, Department of Computer Science and Engineering, Srinivasa Ramanujan Centre, for her encouragement during our project work.

Our guide, **Dr. V. Kalaichelvi**, Associate Professor, Department of Computer Science and Engineering, Srinivasa Ramanujan Centre, was the driving force behind this whole idea from the start. Her deep insights in the fields and invaluable suggestions helped us in making progress throughout our project.

We also thank the project review panel members, **Dr. K. Raghuraman** and **Dr. K. Venkatesh**, for their valuable comments and insights which made this project better.

We would like to extend our gratitude to **Dr. N. Moratanch**, Assistant Professor -III, Project Coordinator and **Dr. S. Suganya**, Assistant Professor -II, Project Coordinator, Department of Computer Science and Engineering, Srinivasa Ramanujan Centre, for organizing and helping us in the successful completion of the project.

We gratefully acknowledge all the contributions and encouragement from my family and friends resulting in the successful completion of this project. We thank you all for giving us the opportunity to showcase our skills through this project.

List of Figures

Figure No.	Title	Page No.
1.1	DL Architecture	12
1.2	FL Architecture	13
4.1	MLP DL-Confusion Matrix	25
4.2	MLP FL- Confusion Matrix	25
4.3	LSTM DL- Confusion Matrix	26
4.4	LSTM FL- Confusion Matrix	26
4.5	CNN DL- Confusion Matrix	26
4.6	CNN FL- Confusion Matrix	26
4.7	DNN DL-Confusion Matrix	27
4.8	DNN FL-Confusion Matrix	27
4.9	CNN+LSTM DL-Confusion Matrix	27
4.10	CNN+LSTM FL-Confusion Matrix	27
4.11	Centralized and FL Comparison	28
4.12	Performance Enhancement Analysis	29
4.13	Convergence Analysis	30

ABBREVIATIONS

IDS	Intrusion Detection System
FL	Federated Learning
MLP	Multi-Layer Perceptron
DNN	Deep Neural Network
CNN	Convolutional Neural Network
LSTM	Long Short-Term Memory
TP	True Positive
TN	True Negative
FP	False Positive
FN	False Negative
TPR	True Positive Rate
FPR	False Positive Rate
ROC	Receiver Operating Characteristic
AUC	Area Under Curve
FedAvg	Federated Averaging
FedProx	Federated Proximal Algorithm
IID	Independent and Identically Distributed
Non-IID	Non-Independent and Non-Identically Distributed

ABSTRACT

The increasing complexity of cyber threats necessitates robust and privacy-protective intrusion detection systems (IDS). Network traffic logs are the fundamental input to train an efficient IDS. Unfortunately, these network traffic logs include sensitive information related to users' credentials, financial transactions, and/or company/organization communications; therefore, collecting and processing this type of information in a centralized manner is illegal due to regulatory bodies such as GDPR or HIPAA, and operational risks associated with a single point-of-failure. In contrast, Federated Learning (FL) provides a solution that allows multiple organizations to collaborate in developing AI-based models without transferring the underlying raw data from one organization to another. As part of this work, we provide an extensive comparative evaluation of five deep learning architectures – Multilayer Perceptron (MLP), Long Short-Term Memory (LSTM), Convolutional Neural Network (CNN), Deep Neural Network (DNN), and hybrid CNN + LSTM model – within two distinct frameworks (centralized vs. federated learning) using the widely-used NSL-KDD benchmark dataset which includes 148,520 samples. Additionally, federated learning experiments were conducted with five clients distributed across 20 communication rounds utilizing the FedProx aggregation method. Our results show that federated learning surpassed centralized learning for all architecture types and performance metrics. Specifically, federated LSTMs achieved the highest accuracy rate (97.55%), as well as the highest precision rate (98.37%), compared to all baseline values. Furthermore, our per-class analysis shows high quality attack detection for DoS (F1 = 99.82%) and Probe (F1 = 97.95%) attacks, whereas the ability to identify rare events (i.e., R2L and U2R minor classes) remains a challenge common to the community. Comparisons against six existing state-of-the-art approaches support the competitive nature of our proposed framework. Overall, the use of FL represents a technically superior, privacy-friendly, and deployable framework for implementing distributed IDS.

KEY WORDS: -

Intrusion Detection System (IDS), Federated Learning (FL), FedProx, Deep Learning (DL), Privacy Preserving.

TABLE OF CONTENTS

Title	Page No.
Bonafide Certificate	2
Acknowledgement	3
List of Figures	4
Abbreviations	5
Abstract	6
CHAPTER 1: SUMMARY OF THE BASE PAPER	8
1.1 Introduction	8
1.2 Problem Definition	9
1.3 Literature Survey	10
1.4 Architecture Diagrams	12
1.5 Hardware and Software Requirements	13
1.6 Modules and Description	14
1.7 Results and Discussion	16
CHAPTER 2: MERITS AND DEMERITS OF BASE PAPER	18
2.1 Merits	18
2.2 Demerits	18
CHAPTER 3: SOURCE CODE	19
3.1 Data Preprocessing	19
3.2 Label Mapping	19
3.3 Train-Test Splitting	20
3.4 Feature Engineering	20
3.5 Handling Class Imbalance	21
3.6 Data Reshaping	21
3.7 Centralized Learning	21
3.8 Federated Learning	23
3.9 Model Evaluation	24
CHAPTER 4: SNAPSHOTS	25
4.1 Confusion Matrices	25
4.2 Model Comparison	28
4.3 Performance Enhancement	29
4.4 Convergence Analysis	30
CHAPTER 5: CONCLUSION AND FUTURE PLANS	31
5.1 Conclusion	31
5.2 Future Work	31
REFERENCES	32
CHAPTER 6: APPENDIX	33

CHAPTER 1

SUMMARY OF THE BASE PAPER

Title: Federated Learning for Privacy-Preserving Intrusion Detection in Software-Defined Networks.

Journal Name: IEEE, May 2024

Publisher: Mubashar Raza et al.

Year: 2024.

The title of the base paper is “Federated Learning for Privacy-Preserving Intrusion Detection in Software-Defined Networks.”. This paper was published in the year 2024.

SUMMARY:

The paper talks about a way to keep software-defined networks safe from cyber attacks. Traditional methods used one computer to learn and detect threats but this made the network vulnerable to attacks and raised privacy concerns. This is because these old methods needed to share information, which could lead to data leaks and a lot of communication problems. To solve this the authors created a system where many computers work together to train a model. They used a dataset called Edge-IIoTset to train a neural network model on many computers. Only the model details are shared with a computer, which combines them so no raw data needs to be sent. The model prepares the data by removing duplicates changing it into a code and using PCA. The model can detect types of attacks with eleven different classes. The authors fine-tuned the model to get the results. The test results show that this system works well with accuracy of about 98%. It also has loss, between 0.03 and 0.04 and it converges steadily no matter how many computers or iterations are used. This system works better than centralized methods and keeps data private making it a good solution, for modern network security. The system is scalable, efficient. Protects privacy well.

1.1 INTRODUCTION

We have seen the rapid expansion of our attack surface via cloud based services and Internet connected devices. As a result, we have the need for better tools to defend against these threats. One tool that provides us with some insight to this issue is Intrusion Detection Systems (IDS). IDS systems provide real-time identification of abnormal behavior or malicious activity occurring on the network. Although there have been many advancements in IDS technology, classical IDS technology does not support new or "zero-day" attacks. Therefore, the use of machine learning (ML) and deep learning (DL) technologies have emerged. These technologies allow IDS to learn how to detect anomalies through data [1]. There is however a major drawback when it comes to applying IDS using ML/DL technologies in real world distributed environments. In order for these IDS systems to be effective they need to be trained on large amounts of historical data. However, one problem that occurs is that the data required to train them is extremely sensitive and therefore cannot be shared across organizational boundaries.

For example, network logs typically include user names, session tokens, financial transactions, personal communications and internal architecture information. When you collect data like this for centralized model training there are three types of risk associated with doing so. Those risks are (i) violation of laws and regulations regarding data protection such as GDPR, PDPA and HIPAA. (ii) creating a single point of failure in terms of security because once organizations share their log data then the organization that stores that log data will become the most attractive target for an attacker. (iii) model inversion attacks where an attacker can determine sensitive training data from the gradients of the model that was used to classify their input [8]. Therefore, in addition to what would be technologically desirable (to have global visibility into threats across all participating networks), we now have significant legal and operational hurdles to overcome before we can deploy a privacy preserving IDS — thus privacy preservation is no longer optional, but rather necessary to make privacy preserving IDS deployable. Federated Learning (FL) addresses this privacy concern. Instead of sending the raw data to a central server, each client trains a local model on its own data and sends the model updates (gradients/weight differentials) back to a central server. At the central server, the model updates are combined (aggregated) to create a global model. By doing this, the clients do not expose their raw data to other parties [2]. FL has shown great promise in various domains including health care [9], mobile prediction [10], and detecting financial fraud [11]; however, despite being widely applicable to network intrusion detection, the application of FL to IDS is still largely unexplored due to the potential degradation of convergence caused by the non-identical and independent distribution (non-IID) of traffic patterns across clients [6, 7].

1.2 PROBLEM DEFINITION

The need to secure networks from attackers has become urgent because of the rapid rise in numbers of Internet-based services and networked systems. However, a traditional intrusion detection system - using a centralized approach to machine learning - collects and processes all of the data on a single site before making an detection. While this approach is effective at detecting attacks, there are a number of limitations associated with its use; for example, each time data is collected the privacy of that data is compromised. In addition, the amount of data collected creates very high levels of overhead, and there is added risk that the single point of failure will be compromised. Also, the current network environment generates very large - scale - or distributed - amounts of sensitive data, so there is no way to securely share this raw data or to safely transfer it between different regions/entities. This is exceedingly important to protect sensitive and secure data, and to protect networks from future breaches - particularly enterprises, cloud computing, and critical infrastructure. Another significant concern is the large number of different types of data that are produced from each client or source that generate a single aggregate output of data; these differences in client datasets cause degradation of performance of the centralized model. Additionally, the prevalence of class imbalance poses a challenge to accurately identifying rare – yet still critical – attack types, such as R2L and U2R. The need now is to implement a distributed, efficient/intrusion detection mechanism that adheres to the principles of privacy preservation through no longer sharing of sensitive raw data and handles distributed/heterogeneous sources of data.

1.3 LITERATURE SURVEY

There have been many advances in the research of intrusion detection, from simple signature-based systems to much more sophisticated paradigms based on machine learning (ML) and deep learning (DL). Tavallaee et al. [1] developed the NSL-KDD dataset to serve as an enhanced alternative to the KDD Cup 1999 dataset to better address the major concerns associated with redundancy and class imbalance found in the KDD Cup 1999 dataset. Since then, NSL-KDD has come to be widely used as the de facto standard for benchmarking intrusion detection systems (IDS), and the five-class categorization (Normal, DoS, Probe, R2L, U2R) they provided represents a realistic and challenging testing environment for evaluating IDSs.

i. Machine Learning-Based Methods for IDS

The initial applications of machine learning-based IDS were limited to the application of a number of different machine-learning techniques including support vector machines (SVM), random forests, k-nearest neighbors, and naive bayes. Debar et al. [8] laid down a foundation of taxonomic categories of intrusion-detection techniques. Through ensembling techniques, researchers have shown that ensemble-based IDSs will generally exceed 90% detection rates when applied to the KDD-derived datasets; however, shallow machine learning models cannot capture the underlying temporal dependencies nor hierarchical feature representations in sequential network traffic that are inherently required to detect intrusions. Kasongo [16] demonstrated through a systematic comparison that tree-based ensemble-based methods perform significantly better than traditional SVM-based methods when applied to high-dimensional space representation of network traffic features; however, these methods are also inferior to deep architectures when applied to the multi-class problem on NSL-KDD. Recently, Kasongo [16] developed a framework for IDS using comparisons of multiple RNN-based architectures -- LSTM, GRU, and SRNN -- and used XGBoost-based feature selection on NSL-KDD and UNSW-NB15 datasets; he confirmed that feature reduction is necessary for successful generalization when dealing with high dimensional network traffic data and that LSTM-based models produce superior multi-class detection performance compared to other forms of RNN-based models.

ii. Deep Learning for IDS

Sequential network flow data is suitable for Recurrent Neural Networks (RNNs); specifically, Long Short-Term Memory (LSTM)-based RNNs are highly effective at capturing long-term temporal relationships in sequential network flows. Yin et al. [4] demonstrated that LSTM-based IDSs achieve higher levels of accuracy than do traditional ML-based IDSs on NSL-KDD; they achieved 97.40% detection accuracy. Convolutional Neural Networks (CNNs) are able to treat flow feature vectors as one-dimensional signals that represent spatial hierarchies and allow CNNs to learn hierarchical features through stacked convolutional filters. Vinayakumar et al. [10] designed a deep CNN framework for detecting intrusions in network flows and achieved 96.90% accuracy on NSL-KDD. Many researchers have designed hybrid CNN-LSTM frameworks that combine the ability of CNNs to learn local features and the

ability of LSTMs to model temporal dependencies in sequential flows. Kim et al. [11] demonstrated that a hybrid CNN-LSTM model achieves 97.10% accuracy on NSL-KDD and performs significantly better than do single architecture baseline models. Hnamte and Hussain[17] expanded upon this concept and proposed a deeper hybrid architecture called DCNNBiLSTM that combines deep CNN with bidirectional LSTM. They achieved 98.27% multi-class classification accuracy on NSL-KDD, the best reported result for a centralized hybrid deep neural network architecture on this benchmark. Their results demonstrate that bidirectional modeling of temporal dependencies produces statistically significant improvements in performance over unidirectional LSTM modeling. Bukhari et al. [18] extended this approach to apply it to wireless sensor networks by developing a federated SCNN-BiLSTM framework. In addition to demonstrating that federated architectures can successfully implement secure and privacy-preserving intrusion detection without requiring the centralization of sensitive network-flow data, Bukhari et al., also demonstrated that their federated architecture could function effectively in environments where there may be malicious participants attempting to disrupt normal operations. Deep Neural Networks (DNNs) with multiple fully connected layers can achieve competitive performance by performing hierarchical feature abstraction; specifically, Khan’s HCRNNIDS achieved 97.80% accuracy on NSL-KDD [9].

iii. Federated Learning for Network Security

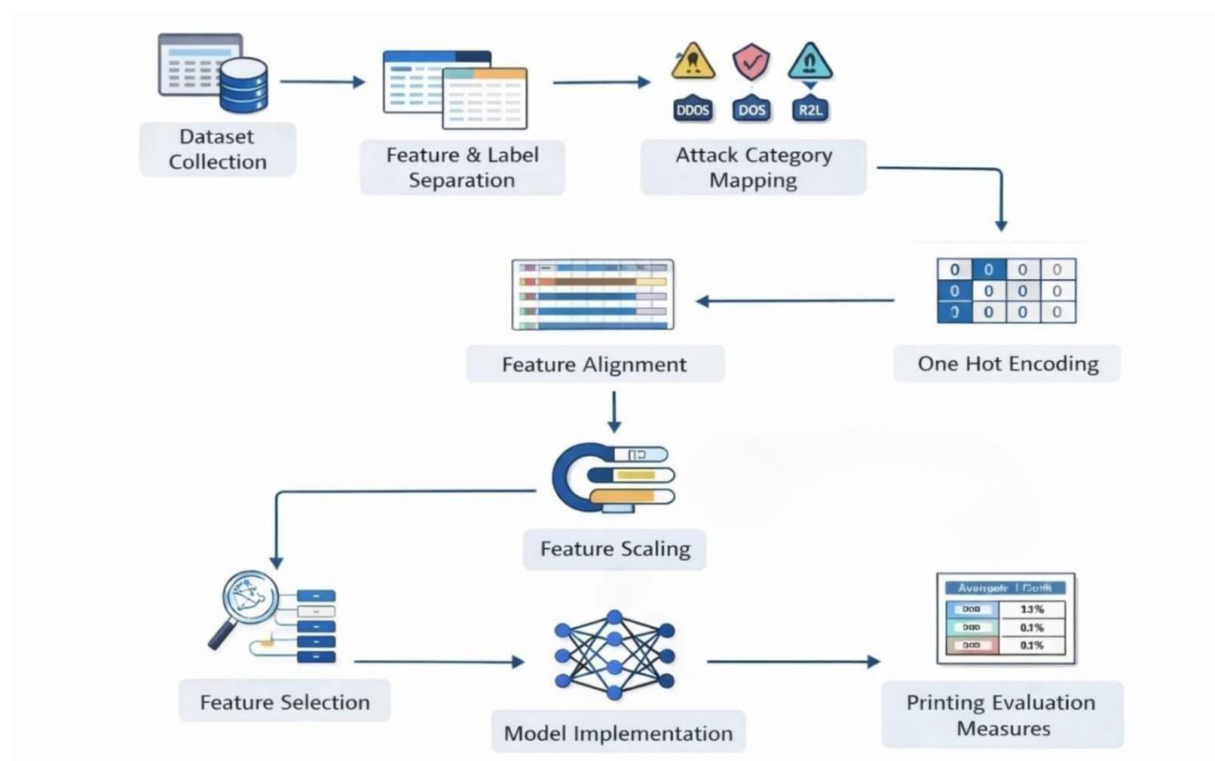
Federated Averaging (FedAvg) was first introduced by McMahan et al. [2]; FedAvg enables distributed training of a global model while maintaining low communication requirements through efficient gradient aggregation. FedProx was later introduced by Li et al. [3]; FedProx extends FedAvg with proximal regularization to constrain parameter divergence between client models and the global model -- crucial for addressing statistical heterogeneity in real-world network traffic. Nguyen et al. [6] conducted a survey of Federated Learning (FL) in cybersecurity and identified IDS as a high priority use case due to the sensitivity of network-flow data and strict regulatory requirements. Mothukuri et al. [12] conducted a review of privacy preserving FL for IoT security and demonstrated that sharing gradients can expose sensitive information about clients through model inversion attacks. However, Rey et al. [13] demonstrated the feasibility of designing a federated IDS for IoT devices that achieved high levels of detection while exhibiting FL robustness to Byzantine failures. Additionally, Al-Marri et al. achieved 98.11% detection accuracy on NSL-KDD using federated mimic learning with MLP backbones. Furthermore, Zhao et al. [14] investigated the non IID problem in FL and demonstrated that even small amounts of globally shared data can greatly improve convergence speed. Finally, Friha et al.'s [19] recent comprehensive review of FL-IDS research contributed during 2017 – 2024 systematically analyzed deep learning based FL-IDS contributions in both network and IoT domains; non IID data handling, communication overhead, and mitigating untrusted participants were identified as the three primary open challenges -- all of which motivated the architecture level comparative study presented here. Most recently, Deshmukh et al. [20] demonstrated that standard FL algorithms can obtain similar results to centralized approaches while dramatically reducing communication overhead; this result supports and validates the empirical evidence presented in section four of this paper.

1.4 ARCHITECTURE DIAGRAMS

Two parallel experimental pathways are evaluated: First, a centralized framework will be tested where all data will reside on one server. Second, a federated learning framework will be tested where all training data is horizontally partitioned across 5 simulated clients.

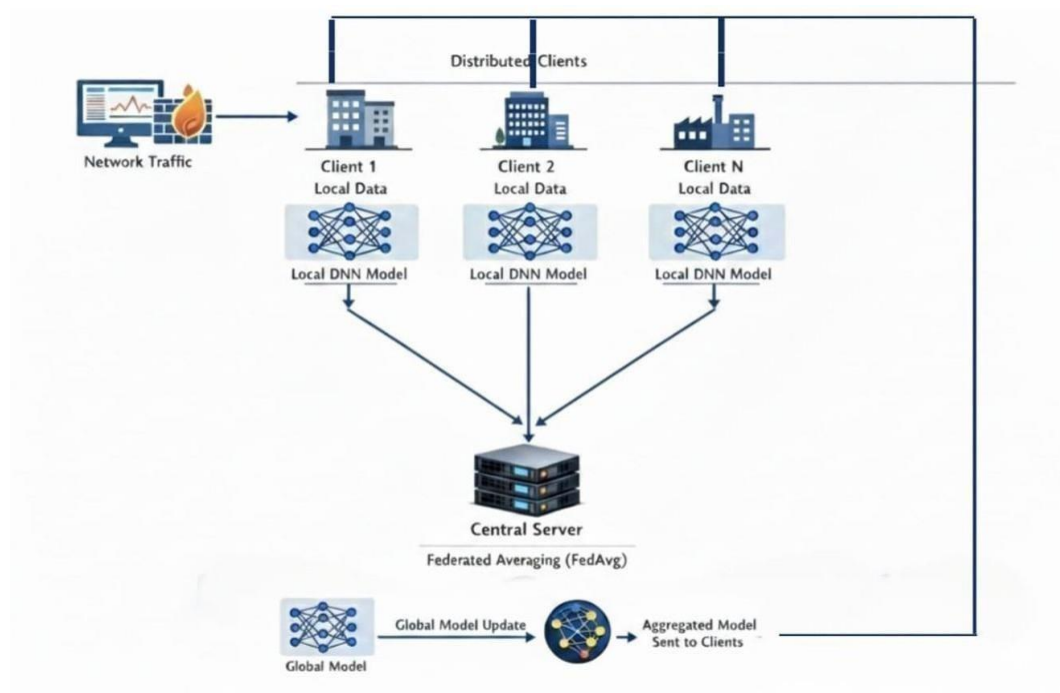
we have evaluated five different deep learning models based on their architectures and capabilities to classify the data. These include Multilayer Perceptron (MLP), Long Short Term Memory (LSTM), Convolutional Neural Networks (CNN), Deep Neural Networks (DNN) and Hybrid CNN + LSTM.

DL ARCHITECTURE



1.1 DL Architecture

FL ARCHITECTURE



1.2 FL Architecture

/

1.5 HARDWARE AND SOFTWARE REQUIREMENTS

The Hardware requirements are

1. Processor : Intel Core i5 or above
2. RAM : 8 GB minimum
3. Hard Disk : 256 GB or above
4. System Type : 64-bit Operating System
5. Internet Connectivity

And the Software requirements are

1. Operating System : Windows/Linux/macOS
2. Front End : Python (Google Colab)
3. Back End : TensorFlow, Keras
4. Libraries : NumPy, Pandas, Scikit-learn, Matplotlib
5. Dataset : NSL-KDD
6. Algorithm : Federated Learning (FedProx), Deep Learning Model

1.6 MODULES AND DESCRIPTION

1.6.1 DATASET DESCRIPTION

The NSL-KDD dataset is made up of 148,520 labeled samples with four classes of attack: DoS (Denial of Service), Probe, R2L (Remote to Local), and U2R (User to Root). In addition, there is a fifth class called "Normal." There are 29,704 samples in the test set and 118,816 samples in the training set. I have used all 41 features of the dataset. All categorical variables were one hot coded and normalized to [0, 1]. I had a very high imbalance ratio with the Normal variable being ~52%, DoS ~36%, Probe ~10%, R2L ~1.5% and U2R ~0.3%. Because of this imbalance ratio, there are many challenges with detecting minority classes.

1.6.2 PREPROCESSING

DEEP LEARNING - DESCRIPTION

Data Collection

Data from different sources is collected to generate network traffic.

In this case, NSL-KDD dataset is used for training and testing purposes.

The data contains both types of traffic, i.e., normal and attack traffic.

Data Integration

All the collected data is then integrated and stored in one system.

This enables all data to be processed in one place.

Moreover, it becomes easy to train the model using complete data set.

Feature – Label Separation

The data set is split into two categories; i.e., feature and label.

Feature represents input data values and describes the characteristics of data.

The label denotes whether the traffic is normal or falls into any of the attack categories.

Attack Category Mapping

Attack traffic is mapped under five major categories, including DoS, Probe, R2L, U2R, and Normal.

It makes classification easier and more systematic.

One-Hot Encoding

One-Hot encoding technique is used to convert categorical values to numerical values.

Binary values are used for each category.

Feature Scaling

Features are scaled such that they fall in a common range.

This helps in reducing the bias created by large values affecting small values.

This improves the performance of the model.

Feature Selection

Relevant features that help in prediction are selected.

Unnecessary features, which are redundant, are omitted.

This results in accuracy and efficiency of the algorithm.

Model Training

All data is utilized in training the model using centralization.

Models like MLP, DNN, CNN, LSTM, and CNN+LSTM are trained.

Training using all the data makes the model capture the global patterns.

Model Evaluation

Scores like accuracy, precision, recall, and F1 are utilized in the evaluation process.

Confusion Matrix is another method used to evaluate model performance.

Prediction

Models that are trained are used to predict classes of test data.

FEDERATED LEARNING – DESCRIPTION

Initialization of clients

Many clients exist during the training process.

Every client has its own datasets which remain unique.

They are exclusive for use within that particular client only.

Global model initialization

The central server initializes a global model.

After that, parameters of that model are sent to all clients.

This model will be used to kickstart the training process.

Local training process

The clients train the model using their individual dataset.

Training is done using different neural networks, e.g., MLP, DNN, CNN, and LSTM.

Learning happens using the client's local dataset.

Model parameter sharing

During the training process, the client does not share its local data.

But instead, the server shares the updated models.

Data security and confidentiality can be maintained.

Model aggregation using fedprox

After completing the local training process, the models are aggregated.

Aggregation process is done using FedProx.

Global model update

The global model is created after combining the model updates by the server.

Again, the updated global model is distributed to all clients.

The above cycle is continued for several rounds.

Training iteration

This continues until a better performing model is obtained.

At each iteration, the model is improved in terms of its accuracy.

Overall, the learning gets improved for all clients.

Evaluation of global model

The model that comes at the end is evaluated using validation/test data.

The performance is calculated using Accuracy, Precision, Recall, and F1 score.

Privacy protection

The original data is always retained on the client side.

Updates to models are shared in the training process.

Thus, privacy is guaranteed by the process

1.7 RESULTS AND DISCUSSION

In this section we will show results of experiments with all 5 deep architectures in both centralized and federated training modes that were tested using the holdout testing set of 29,704 examples from the NSL-KDD dataset. We will follow up with a class-by-class breakdown of each model's performance and compare it to what has been reported in prior literature as the current "state of the art".

Centralized Learning Results

Classification performance of the five architectures are presented. In terms of centralized accuracy and F1-score the best performing architecture is the DNN with accuracy = 97.14% and F1-score = 97.46%. The CNN and LSTM architectures have similar performance with an accuracy of 96.73% and 96.72%, respectively. The worst performance was observed for the hybrid CNN + LSTM architecture which had a lower accuracy at 95.32% than either CNN or LSTM due to having a higher number of parameters that need to be trained from a single data source to reach convergence.

Centralized Deep Learning Model Performance on NSL-KDD

MODEL	ACCURACY	PRECISION	F1 SCORE	RECALL
Centralized MLP	0.9592	0.9782	0.9655	0.9592
Centralized LSTM	0.9672	0.9786	0.9711	0.9672
Centralized CNN	0.9673	0.9791	0.9711	0.9673
Centralized DNN	0.9714	0.9808	0.9746	0.9714
Centralized CNN+LSTM	0.9532	0.9691	0.9584	0.9532

Federated Learning Results

This table shows the federated training results by using the Federated Proximal (FedProx) algorithm. Amongst the many different models that trained with Federated Lstm; Federated Lstm was the most accurate with a 97.55% accuracy, while Federated Mlp had a slightly lower accuracy of 97.49%. The remaining models demonstrated significantly lower accuracies than the top two models. These include: Dnn (accuracy = 97.16%), Cnn (accuracy = 97.0%) and Cnn+Lstm (accuracy= 96.42%). In addition, all of the federated models outperformed their centralized counterpart on each of the four metrics.

Federated Learning Model Performance on NSL-KDD (FedProx)

MODEL	ACCURACY	PRECISION	F1 SCORE	RECALL
Federated MLP	0.9749	0.9839	0.9778	0.9749
Federated LSTM	0.9755	0.9837	0.9781	0.9755
Federated CNN	0.9700	0.9819	0.9739	0.9700
Federated DNN	0.9716	0.9830	0.9753	0.9716
Federated CNN+LSTM	0.9642	0.9788	0.9690	0.9642

CHAPTER 2

MERITS AND DEMERITS OF BASEPAPER

2.1 MERITS

Enhances Privacy of User Data

When users leverage federated learning, they can train the model and never share their private data with others. Therefore, a user who uses federated learning is at much less risk of having their information compromised by other clients/parties.

Lower Centralized Dependency

Unlike a traditional centralized IDS architecture that relies on having one central server in the system, federated learning allows a number of different decentralized clients to have their own models without having a single, central point of failure.

Higher Model Generalization

Federated learning allows training the model on many different clients (and their corresponding datasets), thus enabling it to learn from multiple data sources and achieve better generalization of the overall model and accuracy of the detection of all entities in the model.

Better The Use of Heterogeneous Data

Using a FedProx federated learning algorithm can assist in handling non-identical data that are present among the multiple clients of a federated learning system.

Scalability of the Architecture

Because adding additional or new clients is an easy process in a federated learning architecture, scaling the architecture will not require significant changes to the federated learning architecture.

Reduced Communication Load Compared to Transmission of Raw Data

The federated learning architecture only requires communication with clients via the transmission of model update requests rather than transmitting all of the original datasets to the client, thus allowing for increased conservation of bandwidth.

2.2 DEMERITS

High Computational Cost for Clients

The process of training local models by each client can consume a lot of resources on devices with less processing power.

Communications Still Create Latency

Although they are reduced, the amount of time it takes for clients to regularly communicate with the server can still introduce some latency.

Class Imbalance

Many intrusion systems will have many more samples for attacks than others (e.g., R2L and U2R), resulting in a lower detection rate for these classes.

More Complex to Design, Implement, and Debug

Federated learning systems are harder to design, implement, and debug than centralized learning systems.

Convergence Issues

Because the data being used is not IID, it may take a long period of time for the global models to converge and there will likely be fluctuations in the model during training.

Security Issues with Model Updates

Even though the actual data is not being shared, malicious clients could still submit poison updates that would affect the global model.

CHAPTER 3 SOURCE CODE

3.1 DATA PREPROCESSING

Data preprocessing is performed to clean and prepare the dataset for model training.

3.1.1 Data Cleaning

The dataset may contain invalid rows and inconsistent data types. These are handled as follows:

```
data = data[data['duration'] != 'duration']

for col in data.columns:

    if col not in ['protocol_type', 'service', 'flag', 'label']:

        data[col] = pd.to_numeric(data[col], errors='coerce')

data = data.dropna()
```

3.1.2 Feature and Label Separation

The dataset is divided into input features (X) and target labels (y):

```
X = data.drop(columns=["label", "difficulty"])

y = data["label"]
```

3.2 LABEL MAPPING

The dataset contains numerous attack types, which are mapped into five broader categories for

multi-class classification.

```
def map_attack(x):  
    if x == 'normal': return 0  
    elif x in dos: return 1  
    elif x in probe: return 2  
    elif x in r2l: return 3  
    else: return 4  
  
y = y.apply(map_attack)
```

3.3 TRAIN-TEST SPLITTING

The dataset is divided into training and testing sets using stratified sampling:

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, stratify=y, random_state=42  
)
```

Stratification ensures that the class distribution remains consistent across both sets, which is crucial for fair evaluation.

3.4 FEATURE ENGINEERING

3.4.1 One-Hot Encoding

Categorical features are converted into numerical form using one-hot encoding:

```
X_train = pd.get_dummies(X_train, columns=categorical_cols)  
X_test = pd.get_dummies(X_test, columns=categorical_cols)
```

3.4.2 Feature Scaling

Standardization is applied to normalize feature values:

```
scaler = StandardScaler()  
  
X_train_scaled = scaler.fit_transform(X_train)  
  
X_test_scaled = scaler.transform(X_test)
```

This ensures that all features contribute equally to the model.

3.4.3 Feature Selection

Mutual Information is used to select the most relevant features:

```
mi_scores = mutual_info_classif(X_train_scaled, y_train)

threshold = np.quantile(mi_scores, 0.3)

selected_features = X_train.columns[mi_scores > threshold]
```

This reduces dimensionality and improves model efficiency.

3.5 HANDLING CLASS IMBALANCE

The dataset contains imbalanced classes, which may bias the model. To address this, class weights are computed:

```
class_weights = compute_class_weight(

    class_weight='balanced',

    classes=np.unique(y_train),

    y=y_train

)
```

This ensures that minority classes are given appropriate importance during training.

3.6 DATA RESHAPING

The hybrid CNN-LSTM model requires input data in a 3D format. Therefore, the dataset is reshaped accordingly:

```
X_train_sel = X_train_sel.reshape(X_train_sel.shape[0], X_train_sel.shape[1], 1)

X_test_sel = X_test_sel.reshape(X_test_sel.shape[0], X_test_sel.shape[1], 1)
```

3.7 CENTRALIZED LEARNING USING HYBRID CNN-LSTM

In centralized learning, all training data is stored and processed at a single location.

3.7.1 Model Motivation

The hybrid CNN-LSTM model is used to leverage the advantages of both architectures:

3.7.1.1 CNN layers extract spatial patterns from network features.

3.7.1.1 LSTM layer captures temporal dependencies.

3.7.2 Model Architecture

The architecture consists of convolutional layers followed by an LSTM layer and fully connected layers:

```
model = Sequential([  
    Input(shape=(X_train_sel.shape[1], 1)),  
    Conv1D(32, 3, activation='relu'),  
    MaxPooling1D(2),  
    Conv1D(64, 3, activation='relu'),  
    MaxPooling1D(2),  
    LSTM(64),  
    Dropout(0.3),  
    Dense(64, activation='relu'),  
    Dropout(0.3),  
    Dense(5, activation='softmax')  
])
```

3.7.3 Model Training

The model is trained using:

3.7.3.1 Adam optimizer

3.7.3.1 Categorical cross-entropy loss

3.7.3.2 Class weights to handle

```
imbalanced_model.compile(  
    optimizer=Adam(0.0003),  
    loss='categorical_crossentropy'  
    , metrics=['accuracy']
```

```

)

model.fit(

X_train_sel, y_train_cat,

epochs=10,

batch_size=128,

validation_split=0.2,

class_weight=class_weights

)

```

3.7.4 Limitations of Centralized Learning

- 3.7.4.1 Requires centralized data storage
- 3.7.4.1 Raises privacy concerns
- 3.7.4.2 Difficult to scale across distributed systems

3.8 FEDERATED LEARNING USING HYBRID CNN-LSTM

To overcome the limitations of centralized learning, federated learning is implemented. In this approach, data remains decentralized, and only model updates are shared.

3.8.1 Client Data Distribution

The dataset is divided among multiple clients:

```
NUM_CLIENTS = 5
```

Each client trains the model locally without sharing raw data.

3.8.2 Federated Learning Workflow

The process involves:

- 3.8.2.1 Initialize global model
- 3.8.2.2 Distribute model to clients
- 3.8.2.3 Train locally at each client
- 3.8.2.4 Aggregate updates at server
- 3.8.2.5 Update global model

3.8.3 Federated Averaging (FedAvg)

Client updates are aggregated using weighted averaging:

```
def fed_avg(weights_list, client_sizes):  
    total = sum(client_sizes)  
    avg_weights = []  
    for weights in zip(*weights_list):  
        avg_weights.append(  
            sum(w * (size / total) for w, size in zip(weights, client_sizes))  
        )  
    return avg_weights
```

3.8.4 FedProx Optimization

FedProx is applied to handle non-IID data across clients:

```
updated_weights.append(  
    w - MU * (w - w_global)  
)
```

Significance of FedProx:

3.8.4.1 Reduces deviation from global model

3.8.4.2 Stabilizes training

3.8.4.3 Handles heterogeneous data

3.8.5 Training Process

3.8.5.1 Training is performed over multiple rounds (20 rounds)

3.8.5.2 Each client performs local training

3.8.5.3 Global model is updated after each round

3.8.5.4 This iterative process improves convergence while maintaining data privacy.

3.9 MODEL EVALUATION

The model performance is evaluated using standard classification metrics:

- Accuracy

- Precision
- F1 Score
- Confusion Matrix

accuracy_score(y_test, test_pred)

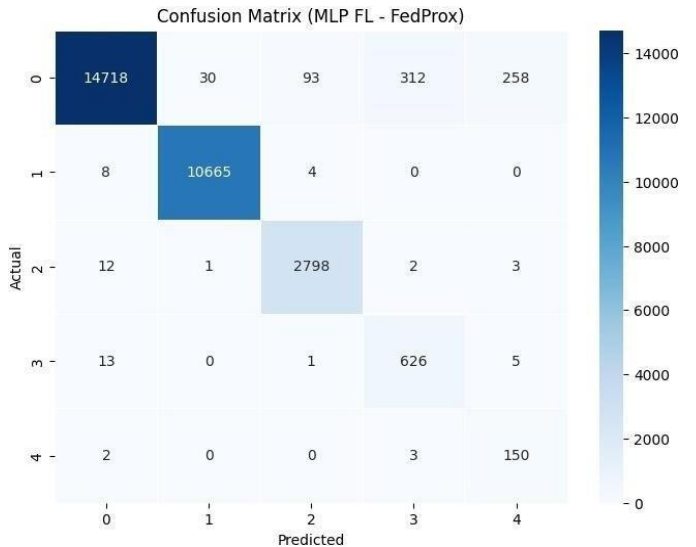
precision_score(y_test, test_pred, average='weighted')

f1_score(y_test, test_pred, average='weighted')

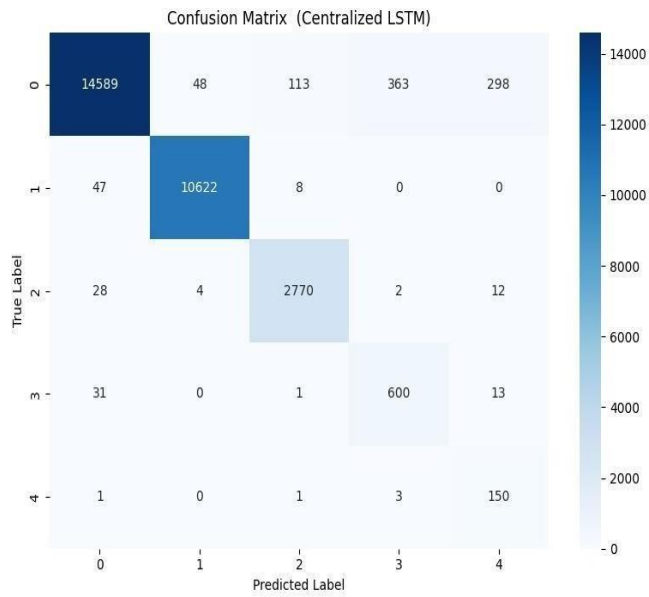
confusion_matrix(y_test, test_pred)

CHAPTER 4 SNAPSHOTS

4.1 CONFUSION MATRIX – CENTRALIZED DL AND FEDERATED DL

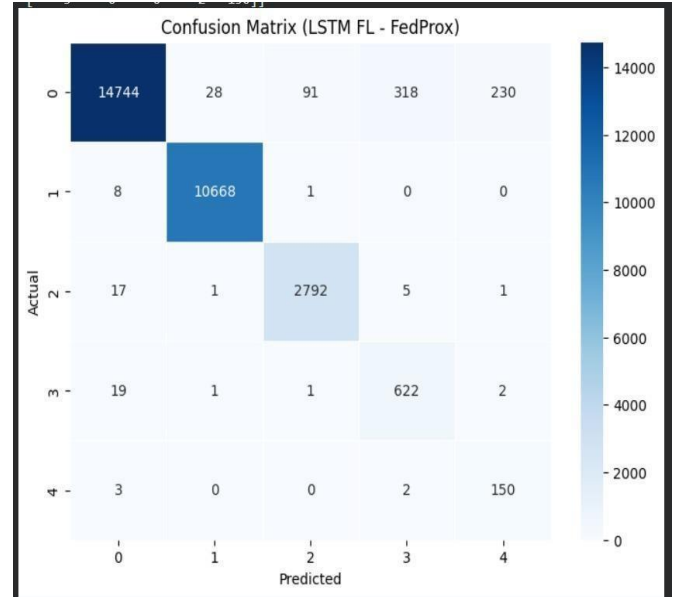
CENTRALISED DL	FEDERATED DL																																																																								
Confusion Matrix - Centralized MLP <table><tr><th>True Label \ Predicted Label</th><th>0</th><th>1</th><th>2</th><th>3</th><th>4</th></tr><tr><th>0</th><td>14316</td><td>44</td><td>130</td><td>583</td><td>338</td></tr><tr><th>1</th><td>12</td><td>10656</td><td>9</td><td>0</td><td>0</td></tr><tr><th>2</th><td>13</td><td>2</td><td>2744</td><td>2</td><td>55</td></tr><tr><th>3</th><td>8</td><td>0</td><td>1</td><td>627</td><td>9</td></tr><tr><th>4</th><td>2</td><td>0</td><td>1</td><td>4</td><td>148</td></tr></table> 4.1 MLP DL- Confusion Matrix	True Label \ Predicted Label	0	1	2	3	4	0	14316	44	130	583	338	1	12	10656	9	0	0	2	13	2	2744	2	55	3	8	0	1	627	9	4	2	0	1	4	148	Confusion Matrix (MLP FL - FedProx) <table><tr><th>Actual \ Predicted</th><th>0</th><th>1</th><th>2</th><th>3</th><th>4</th></tr><tr><th>0</th><td>14718</td><td>30</td><td>93</td><td>312</td><td>258</td></tr><tr><th>1</th><td>8</td><td>10665</td><td>4</td><td>0</td><td>0</td></tr><tr><th>2</th><td>12</td><td>1</td><td>2798</td><td>2</td><td>3</td></tr><tr><th>3</th><td>13</td><td>0</td><td>1</td><td>626</td><td>5</td></tr><tr><th>4</th><td>2</td><td>0</td><td>0</td><td>3</td><td>150</td></tr></table> 4.2 MLP FL- Confusion Matrix	Actual \ Predicted	0	1	2	3	4	0	14718	30	93	312	258	1	8	10665	4	0	0	2	12	1	2798	2	3	3	13	0	1	626	5	4	2	0	0	3	150
True Label \ Predicted Label	0	1	2	3	4																																																																				
0	14316	44	130	583	338																																																																				
1	12	10656	9	0	0																																																																				
2	13	2	2744	2	55																																																																				
3	8	0	1	627	9																																																																				
4	2	0	1	4	148																																																																				
Actual \ Predicted	0	1	2	3	4																																																																				
0	14718	30	93	312	258																																																																				
1	8	10665	4	0	0																																																																				
2	12	1	2798	2	3																																																																				
3	13	0	1	626	5																																																																				
4	2	0	0	3	150																																																																				

CENTRALISED DL

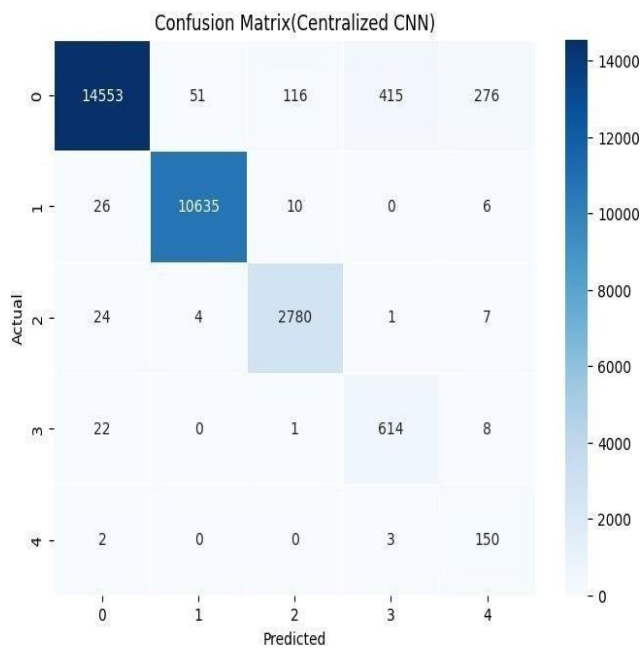


4.3 LSTM DL- Confusion Matrix

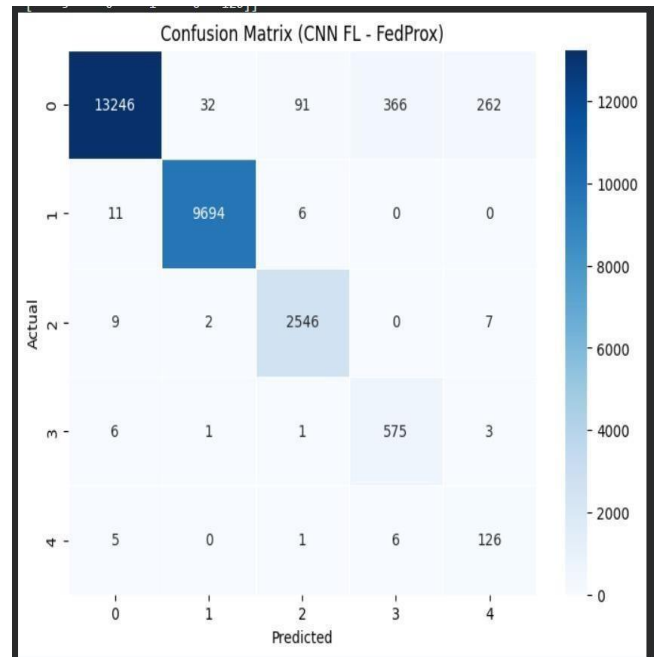
FEDERATED DL



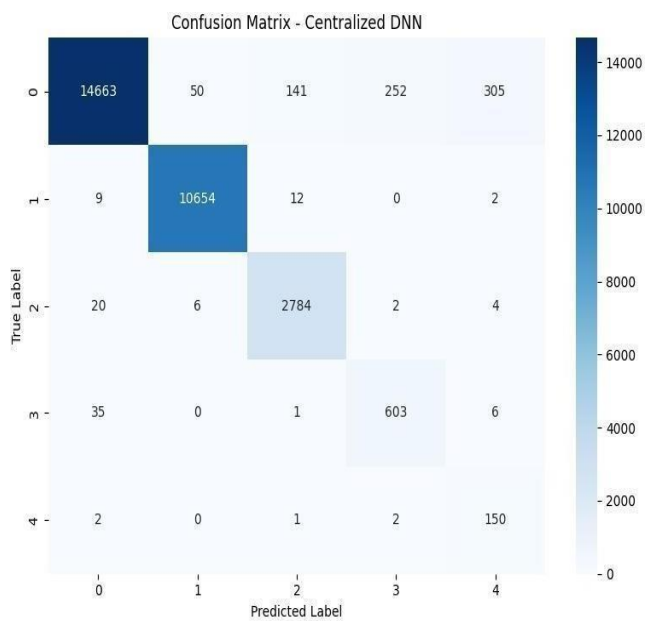
4.4 LSTM FL- Confusion Matrix



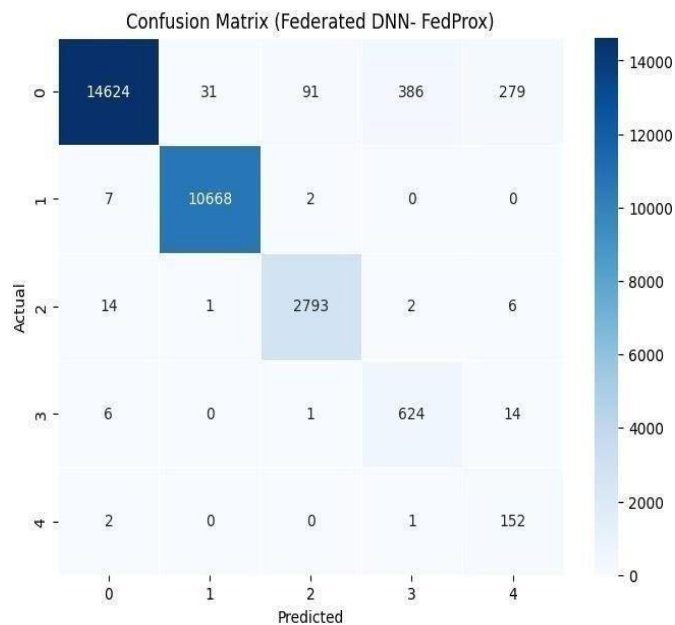
4.5 CNN DL- Confusion Matrix



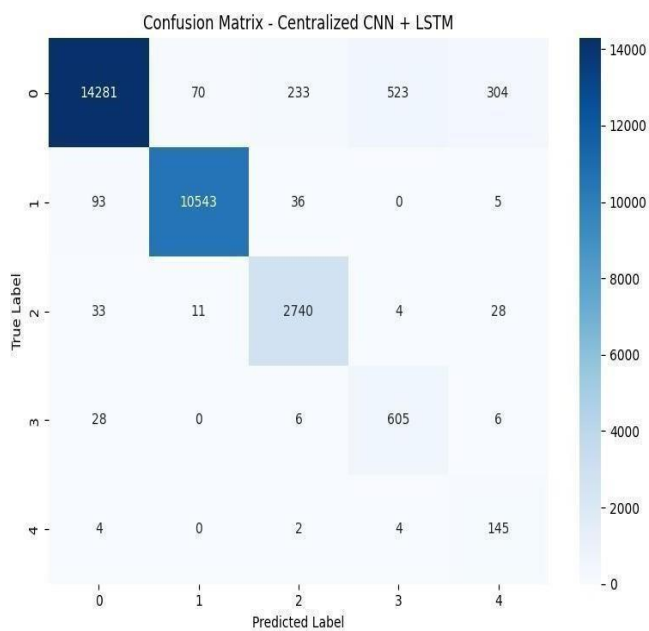
4.6 CNN FL- Confusion Matrix



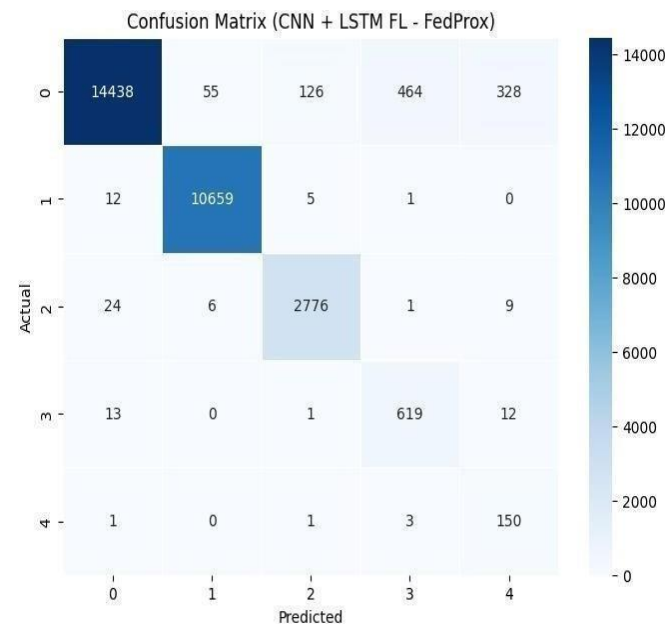
4.7 DNN DL- Confusion Matrix



4.8 DNN FL- Confusion Matrix



4.9 DNN DL- Confusion Matrix

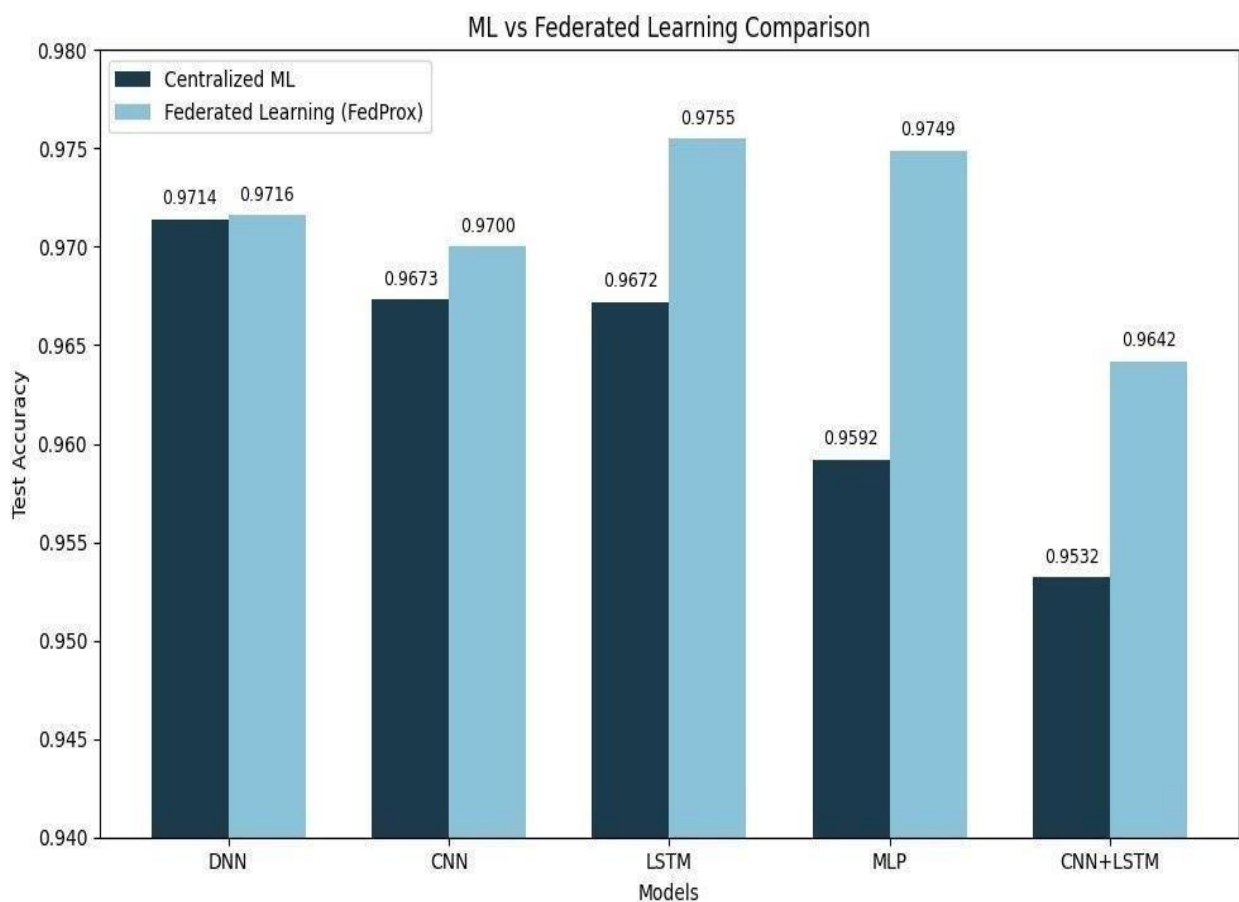


4.10 DNN FL- Confusion Matrix

Analysis of Confusion Matrices:

Confusion matrices help to evaluate how effective the developed intrusion detection models have been created to identify each of the classes based on a comparison of the predicted classes and the actual classes. The entries on the diagonal of the confusion matrices show the instances that have been correctly predicted (i.e., true positives and true negatives) and the entries that are not located on the diagonal of the confusion matrices represent instances that have been incorrectly predicted (i.e., false positives and false negatives). The majority of the instances located on the diagonal of the confusion matrices indicate a high accuracy of the models in predicting the intrusions. The most common classes (Normal, Dos, and Probe) have all been correctly identified with very few errors in their classifications. However, there are some errors in classifying the incorrect class labels of the minority classes (R2L and U2R) resulting from the class imbalance in the NSL-KDD dataset. The results of the comparison between the centralized and federated confusion matrices indicate that the federated model has lower error rates than the centralized model.

4.2 CENTRALIZED AND FEDERATED LEARNING COMPARISON ON MODELS



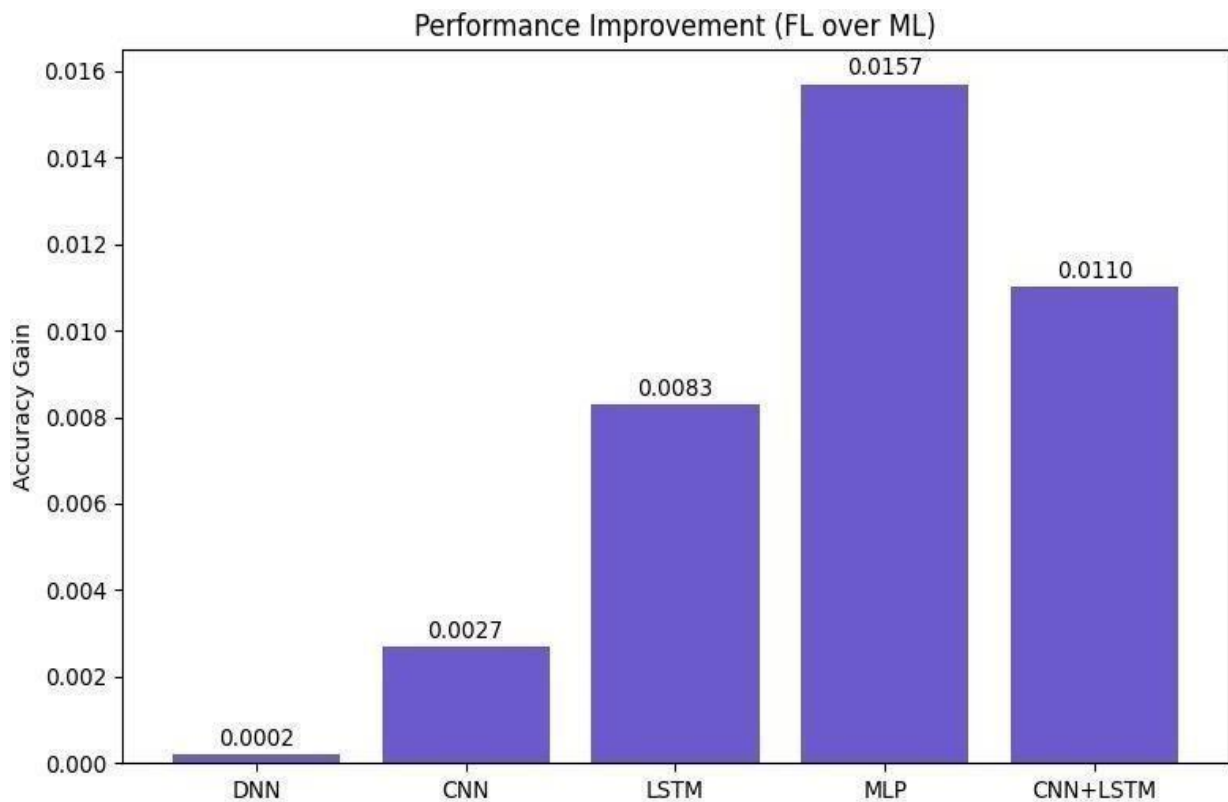
4.11 Centralized and FL Comparison

In the above figure, the test accuracies of deep learning (DL) central models using deep learning (DL) based on the FedProx aggregation method (federated learning) is depicted. It can be seen that, regardless of the model, federated models, on average, provided at least equal or superior performance relative to central models.

The best model overall, the Federated LSTM, achieved a test accuracy of 97.55%; the second best performance was recorded by the Federated MLP (97.49%) indicating they are superior intrusion detection capabilities. There was a small improvement to the DNN, while there was a noticeable improvement to the CNN (Convolutional Neural Network), LSTM e.g., Long Short-Term Memory, MLP, and CNN with LSTM under federated training.

Ultimately, the results demonstrate that federated learning not only allows preservation of privacy, but it also increases classification performance when compared to DL-based, conventional central learning solutions.

4.3 PERFORMANCE ENHANCEMENT ANALYSIS OF FL OVER DL



4.12 Performance Enhancement Analysis

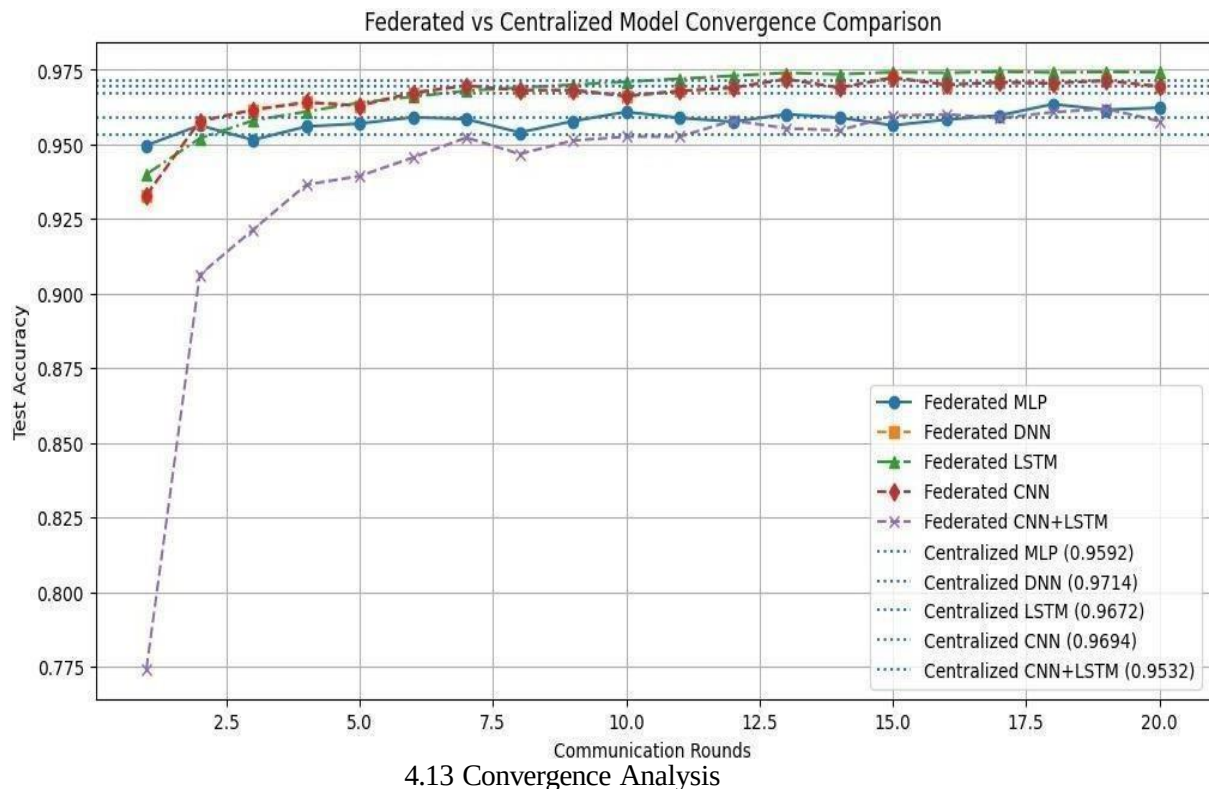
The presented graphs indicate the accuracy improvements made by Federated Learning when compared to Centralized Learning on each individual model. Each model has produced overall positive accuracy improvements and indicates that the Federated Learning approach that was presented works.

The greatest amount of improvement for all models was seen with the Multi Layer Perception (MLP) at 0.0157, then continuing to Convolutional Neural Network with Long Short Term Memory (CNN + LSTM) with an accuracy improvement of 0.0110, and Long Short Term Memory (LSTM) with an accuracy improvement of 0.0083; thus, providing strong evidence of

the added performance benefit of using Federated Learning compared to using Centralized Learning. CNN had a moderate amount of improvement while the Deep Neural Networks (DNN) showed only a very small improvement; however, DNN's had very accurate performance when using Centralized Learning.

These improvements provide strong evidence that the FedProx based Federated Learning framework provides an increase in model generalization and an increase in intrusion detection accuracy while still providing a method for privacy preserved, distributed learning.

4.4 CONVERGENCE ANALYSIS



The federation's performance over time is illustrated in the convergence graph by the number of rounds of communication that have happened. In the first few rounds of communication, performance increased significantly (increased dramatically), whereas performance no longer improves very much (remains fairly unchanged) for the remainder of the communication rounds. The global model appears as if it is converging based on the upward trend of the accuracy or downward trend in the loss function. Once convergence occurs (reaches stabilization), other than having a very small degree of variability, indicates that using FedProx has provided a consistent and effective method to learn from distributed datasets. The results of this research provide evidence demonstrating that the proposed system is capable of learning effectively regarding both robustness and the ability to generalize well.

CHAPTER 5

CONCLUSION AND FUTURE PLANS

5.1 CONCLUSION

In this study we examined a complete comparative evaluation of the effectiveness of five deep learning architectures -- MLP, LSTM, CNN, DNN, and Hybrid CNN+LSTM for network intrusion detection using the NSL-KDD Benchmark Dataset. In addition, we also evaluated each of these architectures using two different machine learning paradigms (centralized and federated) for evaluating the architectures. We found four major points from our experiments. First, federated learning provides similar or better results than centralized learning when using either accuracy, precision, F1 score, and recall as metrics. In particular, we were able to show that Federated LSTMs achieved an overall accuracy of 97.55% which exceeded the best centrally trained model (the DNN at 97.14%). At the same time, we ensured that none of the raw network traffic records left any one of the individual nodes.

Second, the FedProx aggregation method has been very successful in stabilizing the federated convergence process even though there is a large disparity in the distribution of the data. Consequently, we have shown that all five architectures can achieve some form of stable, plateau-like performance by round 20. Additionally, we found that all of the federated models surpassed their respective centrally-trained baselines after 3-5 rounds. These findings confirm both the superior performance and communication efficiency of federated learning versus centralized learning.

Third, in per-class analysis, we found that DoS and Probe attacks are consistently identified ($F1 > 96.7\%$), however, we still found significant challenges with detecting R2L and U2R minority classes in both paradigms. Specifically, we believe that class imbalance will continue to be a field-wide limitation for IDSs in the future -- therefore future work should focus on addressing this issue through augmentations and/or cost-sensitive learning strategies.

Fourth, comparing our proposed federated-learning based approaches with six previously-published state-of-the art approaches demonstrated that the proposed federated-learning approaches are competitive with the current research frontier. Furthermore, our proposed multi-architecture comparative framework provides additional scientific value beyond just demonstrating competitiveness. Our framework demonstrates how a practitioner could select architectures for use in an FL-IDS environment based on evidence obtained from empirical evaluations of multiple architectures. Therefore, the primary contributions of our work is providing a comparative evaluation of architectures in a federated paradigm, providing evidence-based selections of architectures for use in FL-IDS environments, and identifying areas where future work needs to occur.

5.2 FUTURE WORK:

There are several potential avenues for future research based upon our findings. Some possible avenues for future work include: (i) Evaluating our frameworks using Dirichlet distributions to simulate realistic organizational traffic heterogeneity rather than IID data partitions; (ii) Incorporating differential privacy into our frameworks using Gaussian noise added to gradient values in order to provide formal privacy guarantees against model inversion attacks; (iii) Extending our frameworks to utilize three new benchmarks (CICIDS-2017, UNSW-NB15, and CIC-IoT-2023) so that we may assess generalization across different datasets; (iv) Applying

SMOTE oversampling techniques along with cost sensitive loss functions and class weighted FedProx aggregation to improve detection rates for R2L and U2R minority classes; and (v) Developing asynchronous federated learning protocols for clients with varying computational capabilities and/or those whose networks experience periods of disconnection.

REFERENCES

1. [1] Tavallaee, M., Bagheri, E., Lu, W., & Ghorbani, A. A. (2009). A detailed analysis of the KDD CUP 99 data set. *Proceedings of the 2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA)*, Ottawa, Canada, 1–6.
2. [2] McMahan, B., Moore, E., Ramage, D., Hampson, S., & Agüera y Arcas, B. (2017). Communication-efficient learning of deep networks from decentralized data. *Proceedings of the 20th AISTATS*, PMLR 54, 1273–1282.
3. [3] Li, T., Sahu, A. K., Zaheer, M., Sanjabi, M., Talwalkar, A., & Smith, V. (2020). Federated optimization in heterogeneous networks. *Proceedings of the 3rd MLSys Conference*, Austin, TX.
4. [4] Yin, C., Zhu, Y., Fei, J., & He, X. (2017). A deep learning approach for intrusion detection using recurrent neural networks. *IEEE Access*, 5, 21954–21961.
5. [5] Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.
6. [6] Nguyen, D. C., Ding, M., Pathirana, P. N., Seneviratne, A., Li, J., & Poor, H. V. (2021). Federated learning for internet of things: A comprehensive survey. *IEEE Communications Surveys & Tutorials*, 23(3), 1622–1658.
7. [7] Lyu, L., Yu, H., & Yang, Q. (2022). Threats to federated learning: A survey. *IEEE Transactions on Neural Networks and Learning Systems*.
8. [8] Debar, H., Dacier, M., & Wespi, A. (1999). Towards a taxonomy of intrusion-detection systems. *Computer Networks*, 31(8), 805–822.
9. [9] Khan, M. A. (2021). HCRNNIDS: Hybrid convolutional recurrent neural network-based network intrusion detection system. *Processes*, 9(5), 834. MDPI.
10. [10] Vinayakumar, R., Alazab, M., Soman, K. P., Poornachandran, P., Al-Nemrat, A., & Venkatraman, S. (2019). Deep learning approach for intelligent intrusion detection system. *IEEE Access*, 7, 41525–41550.
11. [11] Kim, J., Kim, J., Kim, H., Shim, M., & Choi, E. (2020). CNN-based network intrusion detection against denial-of-service attacks. *Electronics*, 9(6), 916. MDPI.
12. [12] Mothukuri, V., Parizi, R. M., Pouriyeh, S., Huang, Y., Dehghantanha, A., & Srivastava, G. (2021). A survey on security and privacy of federated learning. *Future Generation Computer Systems*, 115, 619–640.
13. [13] Rey, V., Sánchez, P. M. S., Celdrán, A. H., & Bovet, G. (2022). Federated learning for malware detection in IoT devices. *Computer Networks*, 204, 108693.

14. [14] Zhao, Y., Li, M., Lai, L., Suda, N., Civin, D., & Chandra, V. (2022). Federated learning with non-IID data. arXiv preprint arXiv:1806.00582. ICLR 2022 Workshop.
15. [15] Zhang, J., Chen, B., Cheng, X., Binh, H. T. T., & Yu, S. (2022). PoisonGAN: Generative poisoning attacks against federated learning in edge computing systems. IEEE
16. [16] Kasongo, S. M. (2023). A deep learning technique for intrusion detection system using a recurrent neural networks based framework. *Computer Communications*, 199, 113-125.
17. [17] Hnamte, V., & Hussain, J. (2023). DCNNBiLSTM: An efficient hybrid deep learning-based intrusion detection system. *Telematics and Informatics Reports*, 10, 100053.
18. [18] Bukhari, S. M. S., Zafar, M. H., Abou Houran, M., Moosavi, S. K. R., Mansoor, M., Muaaz, M., & Sanfilippo, F. (2024). Secure and privacy-preserving intrusion detection in wireless sensor networks: Federated learning with SCNN-BiLSTM. *Journal of Cloud Computing*, 13, Article 86.
19. [19] Friha, O., Ferrag, M. A., Shu, L., Choo, K.-K. R., Nafis, M. T., & Kim, S. (2025). A review of federated learning applications in intrusion detection systems. *Computer Networks*, 260, 110826.
20. [20] Deshmukh, A., Esteva de la Rosa, P., Dasari, S., & Rodriguez, R. (2025). Enhancing privacy in IoT-enabled digital infrastructure: Evaluating federated learning for intrusion and fraud detection. *Sensors*, 25(10), 3043.
21. N. A. Al-Athba Al-Marri, B. S. Ciftler and M. M. Abdallah, "Federated Mimic Learning for Privacy Preserving Intrusion Detection," *2020 IEEE International Black Sea Conference on Communications and Networking (BlackSeaCom)*, Odessa, Ukraine, 2020, pp. 1-6, doi: 10.1109/BlackSeaCom48709.2020.9234959.
22. S.A. Rahman, H. Tout, C. Talhi, A. Mourad Internet of Things intrusion detection: Centralized, on-device, or federated learning? *IEEE Netw.*, 34 (6) (2020), pp. 310-317, [10.1109/MNET.011.2000286](https://doi.org/10.1109/MNET.011.2000286)

CHAPTER 6

APPENDIX

7.1 Dataset Description

The NSL-KDD dataset is used for training and testing the intrusion detection systems.

- Total number of samples: 148,520
- Number of features: 41
- Classes: Normal, DoS, Probe, R2L, U2R

The dataset is imbalanced, with only a small number of instances in the R2L and U2R classes.

7.2 Preprocessing Procedure

The preprocessing steps include:

- Removal of invalid entries and duplicate records
- Conversion of categorical features using One-Hot Encoding
- Application of feature scaling
- Selection of important features using Mutual Information
- Handling class imbalance using class weights

7.3 Model Setup

Centralized Learning:

- Optimizer: Adam
- Number of epochs: 10
- Batch size:

128 Federated

Learning:

- Number of clients: 5
- Number of rounds: 20
- Algorithm used: FedProx

7.4 Evaluation Criteria

Model performance is evaluated using the following metrics:

- Accuracy
- Precision
- Recall
- F1 Score
- Confusion Matrix

7.5 Important Findings

- Federated learning performs better than centralized learning
- Data privacy is maintained as raw data is not shared
- Minority classes such as R2L and U2R are more difficult to classify

6.6 Software Environment

The project is implemented using the following tools and libraries:

Python, TensorFlow, Keras, NumPy, Pandas, Scikit-learn, and Matplotlib

6.7 Conclusions

This appendix provides additional details about the dataset, preprocessing steps, model configuration, and evaluation methods used in the project.

Received 18 April 2024, accepted 28 April 2024, date of publication 2 May 2024, date of current version 23 May 2024.

Digital Object Identifier 10.1109/ACCESS.2024.3395997

RESEARCH ARTICLE

Federated Learning for Privacy-Preserving Intrusion Detection in Software-Defined Networks

MUBASHAR RAZA^{1,2}, MUHAMMAD JASIM SAEED¹, MUHAMMAD BILAL RIAZ^{3,4},
AND MUHAMMAD AWAIS SATTAR¹

¹Department of Computer Science, Riphah International University, Lahore Campus, Lahore, Pakistan

²Department of Computer Science, Riphah International University, Sahiwal Campus, Sahiwal, Pakistan

³IT4Innovations, VSB—Technical University of Ostrava, Ostrava, Czech Republic

⁴Department of Computer Science and Mathematics, Lebanese American University, Byblos, Lebanon

Corresponding author: Muhammad Bilal Riaz (Muhammad.bilal.riaz@vsb.cz)

This article has been produced with the financial support of the European Union under the REFRESH – Research Excellence for Region Sustainability and High-tech Industries project number CZ .10.03.01/00/22_003/0000048 via the Operational Program Just Transition.

ABSTRACT Software-defined networking (SDN) is an innovative network technology. It changed the world of computer networking by providing solutions to many challenges. SDN provides programmability, easy and centralized network management, dynamic configuration, and improved security. Although SDN offers remarkable benefits but it provides centralized network management which is prone to attacks. So, intrusion detection systems (IDS) are essential to detect and prevent security attacks in SDN. Traditional IDS follow a centralized machine learning approach which causes vulnerabilities in IDS. Old-style IDS lack data privacy preservation, and solution for training data unavailability due to privacy. Federated learning (FL) is a distributed machine learning approach which provides a collaborative training approach without data sharing. In FL, training is performed on multiple nodes creating a global model without sharing the data. To address challenges and the limitations of traditional IDS, we proposed a FL based multi class classification IDS for SDN. FL delivers an efficient and scalable solution to address challenges of traditional IDS. The proposed model enhances security of SDN by not requiring the centralization of data. To test the impact and efficiency of proposed model, we used a latest and realistic cybersecurity dataset. We also compared the proposed model with state of art existing multi class classification studies. The results and their comparison with existing studies highlight the potential of proposed model to enhance network security while providing a privacy-preserving learning environment for intrusion detection.

INDEX TERMS Federated learning, intrusion detection, network security, software defined networks.

I. INTRODUCTION

SDN is a fascinating network technology. It solved many problems and challenges of traditional networks. It is a network management approach that provides many benefits. Although it is a major advancement in network management technology [1]. These days networks are growing unpredictably as a result of increasing number of devices on the Internet [2]. SDN rescued the computer networks for their growing demands and needs. SDN improved the security and

performance of computer networks [3]. It added a factor of ease to managing networks. It offers a software-based centralized network management approach that makes computer networks dynamic, easy to configure, programmable, efficient, and reliable [4]. We can easily monitor networks and their traffic using SDN. Although SDN has many benefits, it also has some vulnerabilities [5]. Those vulnerabilities revolve around the centralized approach.

SDN consist of a layered architecture and it is divided into three planes. The first plane of SDN is application plane. The second plane of SDN is control plane, which is also known as control layer. The third plane of SDN is data plane

The associate editor coordinating the review of this manuscript and approving it for publication was Jad Nasreddine¹.

which is also known as infrastructure layer [3]. Each plane of SDN faces cyberattacks. The control plane is vital because it consists of a SDN controller, a central intelligence of SDN. SDN is a prominent technology that splits forwarding process of network packets from the routing process. The control plane of SDN is considered a brain, so in most cases, attackers try to hijack it [2]. If attackers somehow hijack the control plane, they can control the entire network.

Machine Learning proved helpful against cyberattacks in the last few decades [6]. These days machine learning is a prominent solution in cybersecurity. In this era, the number of devices has increased, and they are still increasing a huge number [6]. As the digital world grows, so does the number of cyberattacks. So, we need an automotive solution against cyberattacks. Machine learning provides this kind of solution. Machine learning can identify if an unauthorized device is trying to connect network. Machine learning can detect if a device has become a zombie. Machine learning can detect a new form of malware using previous signatures [7]. Machine learning can also be used to recommend security policies.

FL is a decentralized machine learning approach. It is a machine learning technique in which we train models on decentralized devices or servers. Training models on decentralized devices are performed without sharing the training data [8]. Each contributor has its local training data. FL is opposite of centralized machine learning, in which all the training data is gathered on a centralized server, and model training is performed. Instead of sharing the data, contributors share the weights of the model [9]. Weight sharing is performed in the iterations until the contributors get the best accuracy. FL permits multiple contributors to build a common model, which is also known as a global model. This global model contains the efficiency of all the contributors. FL solves many data-related issues, including data privacy, unavailability of training data due to privacy and unauthorized data access [10]. These days privacy and data security are prominent issues. Data privacy is preserved by FL [11].

In computer networking, we use IDS to deal with cyberattacks. An IDS monitors computer networks and takes predefined actions if it finds an anomaly in network traffic [12]. An IDS can be a dedicated hardware, software application, or combination [13]. The activity of IDS can vary as per the IDS category and the organization's requirements. Generally, an IDS reports unusual and malicious activity in the network to the administrators. Some IDSs are configured so that they try to prevent attacks; in this case, this kind of IDS is termed an intrusion prevention system (IPS).

We used a dataset [14] named as Edge-IIoTset based on cyber security to test the proposed model. Edge-IIoTset data set was specially created for IDS. According to authors, the data set can be used for FL. The authors designed a specific system to generate and capture network traffic. The designed system contains specific layers, including a SDN layer, network layer, IoT layer and edge computing layer. Edge-IIoTset consists of 61 input features and 2 target features. 61 input features consist of traffic data. First target

variable is named as attack label. The attack label variable consists of two values, either 0 or 1. 0 indicates normal traffic, whereas 1 indicates intrusion. The second target variable is named as attack type. The attack type has fourteen classes of attack including MITM, Uploading, Ransomware, SQL_injection, Port_Scanning, DDoS_HTTP, DDoS_TCP, Password, Vulnerability_scanner, XSS, Backdoor, Fingerprinting, DDoS_UDP, and DDoS_ICMP.

A. RELATED WORK

Priyadarsini and Bera [3] called SDN the future of enterprise networks. The Authors said SDN is a dynamic approach to network configuration. They said SDN faces many server-based attacks due to its centralized control approach. DoS, spoofing, intrusion, and policy violation are some examples. They suggested that a dynamic security enforcement mechanism can help to deal with these attacks. Karmakar et al. [2] said that despite all the benefits, security of SDN remains an open issue. They discussed that a centralized architecture makes it vulnerable to attacks. Cited research paper proposed a threat model to deal with the attacks. In the control plane, a software error in the controller, a malicious application running on SDN controllers, topology poisoning, and threats from the other network devices can cause a serious security threat to SDN.

Apruzzese et al. [7] said that flooding attacks are one of the easiest attacks launched on networks. The attackers send low-rate traffic toward the control plane, which seems to be legitimate. In case attack becomes successful, then this causes a disconnection between control plane and data plane. They claimed that only IDS could prevent this attack. Novaes et al. [15] highlight different security attacks in SDN, which includes DDoS attacks and Portscan attacks. They proposed a mitigation system for these attacks. The proposed mitigation system is a combination of fuzzy and Long short-term memory (LSTM). Lee et al. [1] investigated how SDN architecture fails to prevent the arising security policy issues in the components. They proposed a fuzzy-based testing model to avoid the rising security policy issues among the SDN components.

1) INTRUSION DETECTION SYSTEMS FOR SDN

Haider et al. reported that DDoS attacks are the most common attacks these days. They are rapidly increasing in number on a daily basis [16]. They are a big problem for networks. SDN has a good immune system against security threats, but still, it has many vulnerabilities. DDoS attacks are a big security threat for SDN. The cited research article proposed a security solution against DDoS attacks. To mitigate the DDoS attack threat, the authors proposed an IDS that uses convolutional neural networks (CNNs) and an ensemble approach. They claimed that their proposed IDS is amazing and efficient against DDoS attacks.

Zhu et al. said there are many forms of Residual networks (Resnet) available these days [17]. Each form is an

improvement to the previous version. Residual networks are used in a large number of IDS. Their performance and efficiency are quite impressive. The authors proposed another improved form of residual networks. They claimed that their proposed residual networks have a shorter detection time. They said that previous residual networks used in IDS take undesirable time to detect the attacks. The authors claimed that their proposed IDS has a low detection time.

Phan and Bauschert reported that cyber-attacks cause huge data losses [18]. In SDN, it is quite challenging to mitigate cyber-attacks. SDN improved cyber security as compared to traditional computer networks. SDN proved itself an innovative solution against challenges we faced in the last couple of decades. In SDN, communication protocols are required for communication between data plane and control plane. As control plane is a central intelligence of SDN so it requires mitigation solutions. The cited research paper proposed an IDS based on a double-deep Q-Network. They also proposed an intrusion response policy based on reinforcement learning. They focused on DoS attacks for model evaluation.

Sahoo et al. reported that SDN became a fascinating network architecture in this modern era [19]. It gives a chance to network administrators to take more control over computer networks in such an easy way. The controller of SDN is also known as the operating system of SDN. The centralized controller maintains and runs different applications and network services. Regardless of all these benefits, the controller of SDN is a prime target for attackers. DDoS attacks are growing rapidly in SDN because the control plane is vulnerable. The cited research article proposed an IDS grounded on a Support vector machine (SVM), kernel principal component analysis (KPCA), and genetic algorithms (GA). They tested their model against DDoS attacks.

ElSyed et al. said the SDN fulfills the requirements and demands of today's advanced data centers and enterprise networks [20]. The centralized approach of SDN provides many functions that are beneficial. SDN architecture provides a wide range of paybacks. Risk attacks are a big threat to SDN architecture. If attackers are able to control the control plane of SDN, then they can manage to control and route the traffic in their own way. Network intrusion detection systems (NIDS) can help to secure the SDN architectures. Deep learning is a good approach in NIDS. The cited research article proposed a CNNs-based IDS, which uses a regularizer method to avoid overfitting. Overfitting leads to poor performance of IDS.

Ravi et al. proposed an IDS [21]. This IDS is proposed specifically for SDN-based Internet of Things (IoT) networks. The demand for IoT networks is increasing as the number of IoT devices is increasing rapidly. The authors said an IDS is essential for security of SDN-based IoT networks. In their proposed model, the authors used (kernel-PCA) and gated recurrent units (GRU) to extract the features of data. Later on, they fused the features extracted by both techniques to train the neural networks. They used the final trained NN model for the classification of attacks in the network.

Yang et al. proposed an IDS for SDN [22]. They claimed that their proposed IDS could detect zero-day attacks. They used an unsupervised machine learning approach to build an IDS. They used an ensemble autoencoder to train the unsupervised machine learning model. They claimed that their proposed model has less complexity as compared to the previous NIDS. They also used a privacy preservation frame to protect the data privacy of users. In their proposed privacy-preserving framework, they added noise in the dataset to avoid data leakage and privacy breach of users.

Ahmed et al. said SDN had become part of many IT infrastructures since they were born [23]. Vehicular networks are also using SDN. An SDN-based vehicular network is also known as a software-defined vehicular network (SDVN). Ahmed et al. proposed an IDS and load balancing system for SDVN. For intrusion detection, they proposed an algorithm which is named by them active deep learning. They used a previously available dataset to test and validate their IDS. They claimed that their proposed system is more efficient as compared to the previous ones.

Prathibha et al. reported in their research work [24] that SDN was affected by DDoS, TCP SYN flood, and TCP ACK flood attacks. Another literature by Lanksy et al. [12] reported Brute force, DDoS, Worms, Malware, Web, and browser attacks. Another review paper [25] by Cui et al. found DDoS attacks in different research articles related to intrusion detection in SDN. Rasool et al. [26] reported that the centralized control approach causes serious vulnerabilities, which result in severe attacks. Authors focused on link flooding attacks, and they said it is one of the easiest attacks launched on networks. The attackers send low-rate traffic toward the control plane, which seems to be legitimate. In case this attack becomes successful, then this causes a disconnection between control plane and data plane. They claimed that only IDS could prevent this attack.

2) FEDERATED LEARNING

Data privacy preservation remained an open issue for decades. FL provides data privacy preservation [11]. FL provides a solution that solves the issue of centralized machine learning [27]. Li et al. proposed a smart healthcare system which uses FL for privacy preservation [28]. Their proposed system is a part of IoT. Their proposed system detects Alzheimer's disease. Their proposed system uses voice samples collected from user's IoT voice devices for disease detection. The Internet of medical things (IoMT) is very popular in the healthcare sector. Privacy of patient data is a big issue in IoMT. Han et al. proposed a tele dermatology framework based on FL [29]. In their framework, mobile phones are used to collect images and transfer them.

Ines Feki et al. proposed a FL based approach to classify patients for COVID-19 using chest X-ray images [27]. Fan et al. performed their research on IoMT [30]. They proposed a FL based IoMT framework. Their proposed system provides healthcare services. Guo et al. worked on

TABLE 1. Limitations of previous studies.

Studies	Contribution of works	Limitations of works
Haider et al. [16]	To mitigate the attacks in SDN the main contribution of this research was CNNs and ensemble-based model against DDoS attacks.	This model can only detect DDoS attacks. Privacy of training data is vulnerable.
Sahoo et al. [19]	The main contribution of this research was SVM and GA based IDS against DDoS attacks in SDN.	The proposed model does not provide training data privacy and protection against unauthorized access to data. SVM could cause huge delays in training due to big data processing.
Zhu et al. [17]	This research proposed Resnet based IDS for SDN to classify network traffic in intrusion or normal.	Classification of traffic into intrusion or normal is not enough, we need to know the class of attack for proper mitigation.
ElSayed et al. [20]	CNNs based IDS was the main contribution of this research to classify traffic in SDN in intrusion or normal.	Training data privacy preservation is missing.
Phan et al. [18]	Double deep Q-Networks based IDS was proposed in this research to classify the traffic in SDN in intrusion or normal.	The model could classify only two classes.
Ravi et al. [21]	An IDS for SDN based IoT was proposed using GRU to detect network intrusion.	The data is shared among the IoT devices without taking care of privacy.
Yang et al. [22]	Ensemble Autoencoder was used to detect zero-day attacks in SDN.	Noise is added to training data to avoid data leakage, noise could harm data integrity.
Ahmed et al. [23]	A model named Active Deep Learning was proposed for Software defined vehicular network (SDVN) to detect intrusion.	Privacy of vehicle data is at risk. Data leakage expected.
Ahmad et al. [38]	A model was proposed based on FL for SDN based industrial CPS to detect attacks.	The model could detect six classes only.
Himanshi et al. [39]	A model was proposed to detect attacks on SDN from IoT based consumer devices.	They assigned the same label 1 to 9 classes and 0 to normal class of traffic.
Jie et al. [40]	A collaborative IDS for SDN based vehicular and ad hoc networks.	An IDS which can detect 5 classes of attack.
Zakaria et al. [41]	An FL based mitigation system was proposed for ORAN	A mitigation system which can detect 5 classes of attack in ORAN.
Zakaria et al. [42]	A blockchain and FL based IDS for vehicular edge computing.	An IDS which is evaluated using 10 classes of attack.
Zakaria et al. [43]	An IDS based on FL and explainable AI for IoT networks	Evaluation of model using old dataset.
Zakaria et al. [44]	A network mitigation system based on SDN and FL.	Accuracy achieved by the model is not promising.
Zakaria et al. [45]	A security framework based on FL for industrial IoT.	The proposed model is evaluated using 10 classes of attack.
Bukhari et al. [46]	An FL based IDS for Wireless sensor networks.	The proposed model is evaluated using 5 classes.

computer-aided diagnosis (CAD) [31]. They proposed a medical data processing method that uses FL to take care of medical data privacy. Elayan et al. worked on medical data privacy [32]. Their proposed model uses FL for healthcare data monitoring and analysis.

Driss et al. performed research work on intrusion detection vehicular sensor networks (VSN) using FL [33]. Their proposed system uses a combination of GRU and Random Forest based ensemble techniques. Li et al. proposed a FL based

NIDS [34]. Huang et al. proposed an IDS using FL [35]. They proposed a system for intrusion detection in cyber-physical systems (CPS). Another research article by Zhao et al. proposed [36] an IDS using FL for IoT environments. Liang et al. proposed an IDS to detect cyber-attacks in advanced metering infrastructure (AMI) systems [37]. They used DNN as a training model with FL in their proposed model. According to them, AMI has a vital role in the smart grid (SG), so it is prone to cyber-attacks.

Zainudin et al. [38] proposed FL based IDS for SDN based Industrial CPS, they proposed a multi class model. Babbar and Rani [39] proposed an FL recommender IDS for SDN. Their model detects the attacks coming from IoT based consumer devices. Cui et al. [40] proposed a collaborative IDS for SDN based vehicular and ad hoc networks. They also proposed the multi class detection model.

Houda et al. [41] proposed a FL based jamming attack mitigation system for open radio access network (ORAN). They have tested both binary and multi class approach. Houda et al. [42] proposed a blockchain and FL based IDS for vehicular edge computing. Houda et al. [43] proposed an IDS based on FL and explainable AI for IoT networks. Houda et al. [44] proposed a network mitigation model based on FL, blockchain and SDN. Houda et al. [45] proposed a security framework based on FL for industrial IoT applications. Bukhari et al. [46] proposed an FL based IDS for wireless sensor networks.

B. LIMITATIONS AND DRAWBACKS OF EXISTING STUDIES

Huge amount of data must be collected from different sites and must be sent to a distant central server for model training, which requires data transmissions causing delays. These data transmissions and delays have significant impact on model training. The whole process of traditional centralized model training is prone to attacks and data privacy violations. Data owners of network traffic are worried about the data privacy. Furthermore, this privacy concern of owners causes unavailability of training data. The traditional centralized model training failed to provide data privacy and security. Data transmissions to a central server from different sites can compromise the quality and integrity of data.

Major limitations of previous research works have been summarized in Table 1. The previous research works did not provide privacy of training data. The studies used centralized machine learning approach which has many drawbacks. Some of the previous research work could classify only DDoS attacks and some of them could classify traffic in intrusion and normal only. Previous studies overlooked the problem of big data processing, which could cause delays in training. Data leakage problem was overlooked as well. Some studies of them could classify the intrusion into important classes of attacks. To implement proper mitigation of attacks, identification of cyber-attack class is vital.

Let's compare the existing studies with this research work. The proposed model by Haider et al. [16] can only detect DDoS attacks. Privacy of training data in this model is vulnerable. Whereas the proposed model can detect 11 types of important attacks and provides privacy for training data. The study conducted by Sahoo et al. [19] does not provide training data privacy and protection against unauthorized access to data. SVM could cause huge delays in training due to big data processing. Whereas the proposed model overcomes the challenges of their study by using unique FL and approach. The study proposed by Zhu et al. [17] can only classify the traffic into normal or intrusion. Their study does not

classify the attacks into specific type. Classification into specific type of attacks, helps to mitigate them and prevent them precisely. The proposed work classifies the attacks into specific types, which adds the uniqueness to this research work.

The research conducted by ElSayed et al. [20] does not provide privacy preservation and security to the training data where the proposed model provides the privacy. The model proposed by Phan and Bauschert [18] can classifies the attacks into two classes only whereas the proposed research can classifies important 11 attacks. Model proposed

by Ravi et al. [21] shares the data among the IoT devices without taking care of privacy whereas overcomes this vulnerability. Yang et al. [22] added noise to training data to preserve the privacy but this approach hits the integrity of training data whereas the proposed model preserves the privacy of data without adding any noise. Model proposed by Ahmed et al. [23] shares the vehicle without taking care of any data security whereas the proposed model provided the data security.

Zainudin et al. [38] proposed an IDS which can detect 6 classes of attack whereas the proposed model is evaluated using 11 classes of attack and it achieved higher accuracy than their model. Babbar and Rani et al. [39] proposed an IDS which can detect 10 classes. They assigned the same label 1 to 9 class and 0 to normal class of traffic. So, their model is not able to tell us which type of attack detected. Furthermore, they evaluated their model on old datasets. Whereas the proposed model is evaluated using 11 classes and it can tell us the type of attack. Cui et al. [40] proposed an IDS which can detect 5 classes of attack whereas the proposed model is evaluated using 11 classes of attack and it achieved higher accuracy.

Houda et al. [41] proposed a FL based mitigation model for ORAN. They have evaluated their model using 5 classes of attack. Whereas we proposed an IDS for SDN which can detect 11 classes. Houda et al. [42] proposed an IDS which is evaluated using 10 classes whereas the proposed model is evaluated using 11 classes. Zkaria et al. used an old dataset to evaluate their proposed model whereas this research used a latest and state of art cyber security-based dataset. Houda et al. [44] proposed a network attacks mitigation framework. The proposed model achieved higher accuracy as compared to their model. Houda et al. [45] proposed a security framework for industrial IoT applications. They evaluated their model using 10 classes whereas the proposed model is evaluated using 11 classes and a latest cybersecurity dataset. Bukhari et al. [46] proposed an FL based IDS for wireless sensor networks. They have evaluated their model using 5 classes of attack whereas the proposed model is evaluated using 11 classes. The higher number of classes increases its scope and efficiency.

C. MOTIVATION

In section II of I and B of I, we discussed and observed that FL can be a practical approach to address the issues faced by traditional machine learning based IDSs. After getting

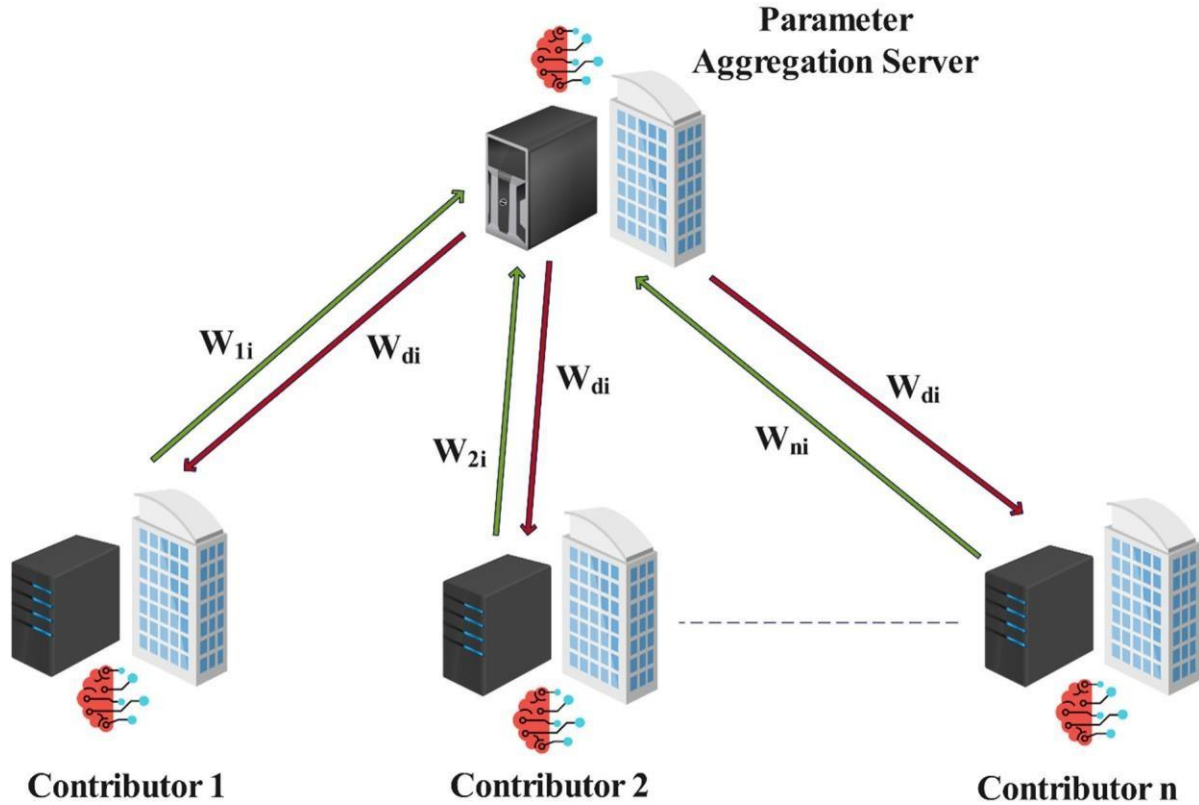


FIGURE 1. System architecture for privacy preserving intrusion detection in software defined networks.

motivation from those observations, this study proposes an FL based IDS, that can be implemented in SDN to protect it from cyber-attacks. There are many advantages of using FL approach, as compared to non-FL approaches [11], [27], [47]. FL provides edge devices-based training. In FL data is saved on local devices which ensures the security of data. FL is an upgradation of traditional machine learning approaches. Experimental results shows that the proposed IDS can be a practical and efficient approach against cyber-attacks.

D. PROBLEM FORMULATION

SDN is an innovative and comparatively new approach to networking. It faces many security challenges. IDS is a good solution to detect and prevent security attacks and breaches in SDN. Traditional IDSs have many flaws and draw backs. We already have discussed those drawbacks in detail in previous sections. To overcome their drawbacks, this study we propose FL based innovative IDS approach. Fig. 1 shows the system architecture and Fig. 2 shows the basic work flow of model. On client side there can be 1 to n contributors. Each contributor has been provided its own local training data. Each contributor trains their local model. After training the contributors sent their parameters to server. Server aggregates the parameters using Equation 13. Then server send the update to contributors. Now, contributors update their parameters. Here first global iteration completes. The process

Algorithm 1 Contributor Side

- 1: Start
- 2: Local data splitting into training and validation
- 3: Initialize the weights and layers of training model
- 4: Initialize the epochs of training model
- 5: for number of epochs of training model
 - i. Apply the feed forward step of ANN
 - a. Calculate $\mathcal{Z}_i$ using equation 1
 - b. Calculate $\hat{R}$ using equation 2
 - ii. Calculate W using equation 5
 - iii. Weights updating step
 - a. Calculate v_{jkl}^{ci} using equation 9
 - b. Calculate using Δw_{ij}^{ci} equation 10
 - c. Update the weights between hidden and output layer using equation 12
 - d. Update the weights between hidden and input layer using equation 11
 - iv. If number of local iterations do not end go to step 5 else go to step 6
- 6: Send optimized wights to aggregation server
- 7: Stop

of training continues until the global model converges. The algorithm for client-side training is given in Algorithm 1 and algorithm for server side is given in Algorithm 2.

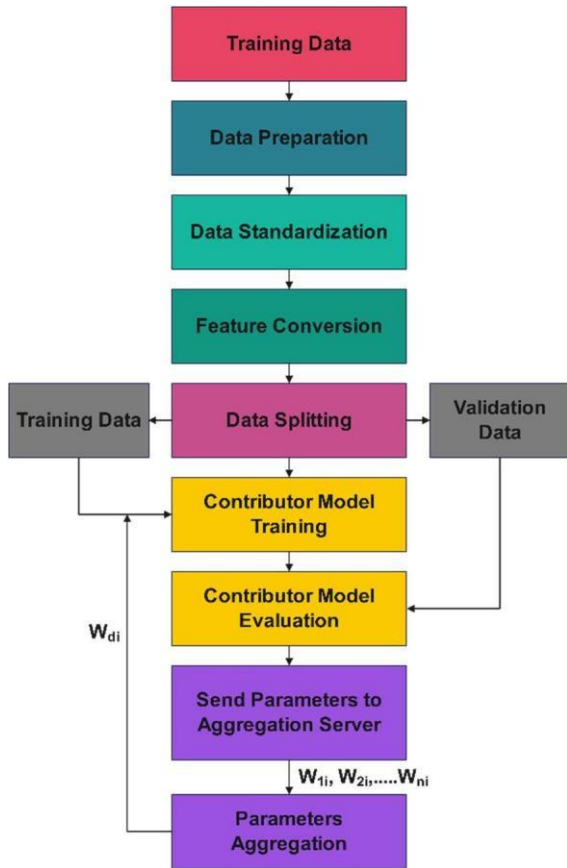


FIGURE 2. Basic work flow of model.

Algorithm 2 Server Side

```

1: Start
2: Initialize server with initial weights
3: for each global iteration
    i. each contributor
        a. Check weights dimension and equalize
        b. Aggregate weights using equation 13
    end
end
4: Perform predictions
    i. For number of validation samples
        a. Calculate  $\mathcal{Z}_i$  using equation 1
        b. Calculate  $\hat{R}$  using equation 2
        c. Calculate  $\psi$  using equation 5
5: Send parameter update to contributors
6: Stop
  
```

E. MAIN CONTRIBUTIONS

The main contributions of this research work are as follows:

- In this research work we implemented FL based IDS to perform multi class classification of security attacks in SDN. The proposed model ensures the data privacy preservation and avoids the data leakage problem by not requiring the transfer of training data on centralized

server. The proposed model also eliminates the requirement of big data processing by not requiring the training data on central place.

- The proposed model is trained and validated using a latest cyber security dataset “Edge-IIoTset: A new comprehensive realistic cyber security dataset of IoT and IIoT applications for centralized and federated learning”. The dataset was created using real time SDN traffic.
- As a baseline model of FL, we proposed an optimized ANN architecture which is simple, efficient and result oriented. The proposed ANN architecture has been evaluated by varying a range of hyperparameters.
- We used PCA to reduce the data dimensionality. Low dimensional data helps to reduce computation power and reduces latency.
- Most of the models proposed in the existing studies could classify binary or single class of attacks. This restriction made their scope and performance limited. The proposed model can classify attacks into 11 important classes. Such classification is beneficial for mitigating attacks. We also compared the proposed model with the latest and state of art existing multi class classification models to explain its effectiveness and novelty.
- We implemented a detailed experimental evaluation using various performance metrics and compared the results with latest existing studies, results show that the proposed model provides better scope and performance.

II. METHODOLOGY

This research work focuses to improve the security of SDN by improving IDS. In this research work, we used an advanced machine learning technique to implement an IDS for SDN. Centralized machine learning algorithms and techniques face many challenges. FL is an advanced machine learning technique which offers a solution to data privacy preservation in machine learning and improves the performance and security. The proposed model solves challenges of centralized machine learning and provides a highly efficient solution for intrusion detection in SDN.

There are few techniques and operations that we have performed to preprocess and clean the dataset. Duplicate records in a dataset are not good for any machine learning model. They badly affect the training and validation of the model. So, we must drop them to improve and maintain the model’s accuracy and avoid misleading performance. First, we checked the null values in the dataset using `isnull.sum()` command and it showed 0 null values in each feature. Then we checked for na values using `isna.sum()` command, and the result was the same; there are no na values in the features. And at the end, we checked the duplicate values using `duplicated-value_counts()` commands, and it showed 235973 duplicate records. So, we dropped the duplicate records using the

`drop_duplicates(subset = None, keep = "first", inplace = True)` command.

Machine learning algorithms do not accept and process categorical data. They require variables in numerical form. Label encoding is a process and method in which we convert categorical variables into numerical form. There are a few techniques and algorithms available in Python that convert the categorical variables into numerical form each one has its pros and cons. We used `LabelEncoder` from `sklearn` to convert categorical variables. It converts categorical data into 0 to $n-1$ integer values. `LabelEncoder` is a simple approach, which was suitable. We assigned labels to classes using `LabelEncoder`.

Later on, we splitted the data into 70% and 30%. 70% data was used for training and the rest of the 30% was used for model validation. The 70% data was equally divided among the contributors for training purpose. The principal component analysis (PCA) is a feature extraction and dimension reduction technique. Principal component analysis reduces the data dimensionality, redundancy and preserves the maximum information [38], [39]. Principal component analysis converts the n features into k principal components by preserving the maximum information. In IDS, a huge amount of data is required for model training which requires high computation power and causes delays in the training process. Principal components are created using a mixture of early variables. Newly created variables or principal components are created in such a way that they have a minimum correlation. There are four classes in the attack-type target variable which has a negligible appearance in the dataset. We dropped those classes to improve the performance of the machine learning model. After applying the PCA and Data pre-processing technique final shape of dataset becomes (1957249, 23).

A. BASELINE MODEL ARCHITECTURE

Each contributor in FL uses a machine learning algorithm as a machine learning model and training algorithm. There are many machine learning algorithms available that can be used in FL as a baseline model. FL is based on play with parameters in the form of weights. So, in FL, parametric methods are preferred as a baseline model. Parametric algorithms provide a facility to learn, adjust and define the training parameters according to the dataset. In this case, we used ANN as a baseline model, which is a parametric method. we created a simulation environment using Python programming to test the proposed model. Before performing the training, we further divide the training data horizontally among contributors. The reason behind horizontal division is that in IDS, we need traffic data for training, and most traffic data have the same features, so it is the best fit for problem at the hand.

The architecture of the ANN baseline model contains 4 layers in total. First comes input layer, then we added two hidden layers, and at last, we added an output layer. The first three layers contain the `relu` activation function, and the output layer contains the `softmax` activation function. We used a sequential model from `TensorFlow` to implement the baseline

model. The sequential model provides the facility to create different kinds of deep learning models containing layers. The input layer contains 64 neurons, first hidden layer contains 64 neurons, second hidden layer contains 32 neurons, and output layer contains 11 neurons.

Let's discuss and cover some details to explain the decision-making process for ANN architecture further. We started with 1 input layer, 1 output layer, and 1 hidden layer. It was the minimum requirement for ANN architecture. The output layer contains 11 neurons; we cannot change that number because it depends upon the number of output classes, and they are 11; in this case, we set the 11 neurons in the output layer. The number of neurons in the input layer is 64. As we discussed before, they should be more than input features; we have 23 input features for training, so first of all, we set the neurons in the input layer to 32. And we set 32 neurons in the hidden layer. At that time, we had 3 layers in total. We tested the model, and it gave an accuracy of around 80.

To further optimize the model, we increased the neurons in the input layer and hidden layer to 64; it boosted the accuracy to up to 95%. After that, we increased the neurons in the input layer and hidden layer to 128 and 128, but they were not practical; then we expanded it to 256 and 256, but the result was the same, it increased the execution and training time of the model it was like diminishing returns. So, we decided to keep it 64 and 64. To further optimize the model, we added another hidden layer with 32 neurons, which increased the model's accuracy by more than 98%. Then we increased neurons in the newly added layer to 64 and then 128, but it was ineffective. We added more hidden layers to the model, but they did not give positive results, so we decided to keep the architecture with 2 hidden layers.

B. HYPERPARAMETERS

Hyperparameters are parameters of deep learning models whose values are adjustable, and they are set before starting the model's training. Before setting the final values of hyperparameters, we set certain values and then train the model; this process is repeated until we get the best result. A very high learning rate causes the model to converge in less time, affecting accuracy. A very small learning rate causes the model to converge too late. So, the learning rate must not be too low or too high. In this case, we used a learning rate of 0.001. We tested the model by setting the learning rate from 0.1 to 0.001. And we got the optimal result on 0.001. We used the Adam optimizer. We also tried some other optimizers, but we found that Adam is the best suitable. We used two activation functions in the baseline model. `Relu` (Rectified Linear Unit) is used in input and hidden layers, whereas `softmax` is used in output layer. We used `softmax` in the output layer due to multi-class problem. Due to multiclass problem, we used `categorical_crossentropy`. The proposed model is different from traditional models due to the FL approach. So, we set 1 epoch for each contributor or local model. We have sufficient data, so we set the batch size to 64. It is the batch

TABLE 2. Simulation results.

Contributors	Global Iterations	Global Accuracy	Global Loss
3	10	98.21%	0.040
3	30	98.65%	0.033
3	50	98.64%	0.031
5	10	98.29%	0.040
5	30	98.43%	0.036
5	50	98.56%	0.033
10	10	98.16%	0.044
10	30	98.39%	0.038
10	50	98.43%	0.036
10	100	98.63%	0.033

size for each contributor. We tried 32, 64, 128, and 256 batch sizes, but only 64 was more effective.

The proposed system is tested by varying the number of global iterations and contributors. Simulation results show that changing the number of contributors does not affect the accuracy and performance of proposed model. More global iterations can give more precise results but we find that 50 to 100 global iterations are enough although they can be varied depending upon the dataset. More details about global iterations and their impact are discussed in section ‘‘IV. Simulation Results’’ and a summary is given in Table 2.

C. MODEL TRAINING

The following steps explain and highlight the overall training process of proposed system.

1) CONTRIBUTOR TRAINING

All the contributors or clients perform their training on provided local data. In the simulation environment, we created a function that trains a set number of contributors and stores their parameters.

2) AGGREGATION SERVER PARAMETERS UPDATE

Each contributor sends weights or parameters to the aggregation server after training.

3) PARAMETERS AGGREGATION

After receiving the parameters from contributors or clients, the aggregation server scales and averages the weights using mathematical operations. We used an algorithm named Federated Averaging (FedAvg) to perform aggregation.

4) PARAMETERS BROADCASTING

After performing the aggregation, the aggregation server broadcasts the averaged parameters to the clients.

5) CONTRIBUTORS UPDATE

The clients update their parameters and test the performance on receiving the parameters or weights. At this step, one common iteration completes.

6) REPEAT

On completing step 5, the system returns to step 1 and repeats until step 5. In this way, required number of common iterations can be performed until model converges.

D. COMMUNICATION ARCHITECTURE

In FL, the communication architecture is given by the FedAvg algorithm [11]. The transmission control protocol (TCP) can be used for sharing the parameters between contributors and the central server. Significant communication is required, and this process is repeated until the model converges, so this process can cause significant communication overhead if the number of clients participating in the FL process is too large. Due to this communication overhead, there can be latency introduced in the training process. The latency depends upon the communication overhead, and communication overhead relies on the efficiency of the communication network. So, a highly efficient communication network can reduce the communication overhead and latency. The purpose of FL is to preserve privacy, although the clients and central server can also be subject to cyberattacks. To minimize this concern, secure aggregation protocols and encryption methods can be used [48].

E. SYSTEM ARCHITECTURE

Earlier we have discussed the training steps of proposed systems. Let’s discuss the proposed systems architecture in further detail using graphical representation. In Figure 1, the graphical model is shown. In Figure 1, W_{1i} , W_{2i} , up to W_{ni} , represent the weights sent by contributors to aggregation server whereas W_{di} , represents the aggregated weights sent by aggregation server to contributors. Controllers and aggregation servers are based on SDN controllers.

1) CLIENT SIDE

In this research, ANN architecture has been used as a baseline model for training. In the ANN architecture four layers has been used in total including one input layer, two hidden layers and one output layer. The first three layers contain the relu activation function, and the output layer contains the softmax activation function. Softmax has been used because the problem being addressed is multi-class problem. The softmax activation function is given in equation 1, where $k = 1, 2, 3, \dots, n$. The relu activation function is given in the equation 2, where $j = 1, 2, 3, \dots, n$. Equation 3 gives the mathematical form of input layer where $j = 1, 2, 3, \dots, n$. Equation 4 gives the mathematical form of output layer where $k = 1, 2, 3, \dots, n$. Equation 5 gives the mathematical form of error calculation, where c_k and $\hat{R}_k$ represent the actual output and calculated output.

$$\mathcal{Z}_i = \frac{e^{(\tilde{N}_i)}}{\sum_{j=1}^n e^{(\tilde{N}_j)}} \quad (1)$$

$$\hat{R}^k = \max(0, \hat{f}_k) \quad (2)$$

$$\tilde{N}_j = n_1 + \sum_{i=1}^r (w_{ij} \times m_i) \quad (3)$$

$$\hat{h}_k = n_2 + \sum_{j=1}^P (v_{jkl} \times S_i) \quad (4)$$

$$W = \frac{1}{2} \sum_k (c_k - R_k)^2 \quad (5)$$

Equation 6 gives the mathematical form of rate of change in weights for the output layer.

$$\Delta v_{jkl}^{ci} = - \in \frac{\partial \psi}{\partial v_{jkl}^{ci}} \quad (6)$$

After performing mathematical operation on equation 6, equation 7 has been obtained.

$$\Delta v_{jkl}^{ci} = - \in \frac{\partial W}{\partial \hat{R}_k} \times \frac{\partial \hat{R}_k}{\partial \hat{h}_k} \times \frac{\partial \hat{h}_k}{\partial v_{jkl}^{ci}} \quad (7)$$

After performing the substitution in equation 7, equation 8 has been obtained.

$$\Delta v_{jkl}^{ci} = \in (c_k - \hat{R}_k) \times \hat{R}_k (1 - \hat{R}_k) \times \hat{R}_j \quad (8)$$

And after substituting ϕ in equation 8, equation 9 has been obtained.

$$\Delta v_{jkl}^{ci} = \in \phi_k \hat{R}_j \quad (9)$$

$$\Delta w_{ijl}^{ci} \propto - \left[\sum_k \frac{\partial W}{\partial \hat{R}_k} \times \frac{\partial \hat{R}_k}{\partial \hat{h}_k} \times \frac{\partial \hat{h}_k}{\partial \hat{R}_j} \right] \times \frac{\partial \hat{R}_j}{\partial \hat{h}_j} \times \frac{\partial \hat{h}_j}{\partial w_{ijl}^{ci}}$$

$$\Delta w_{ijl}^{ci} = - \in \left[\sum_k \frac{\partial W}{\partial \hat{R}_k} \times \frac{\partial \hat{R}_k}{\partial \hat{h}_k} \times \frac{\partial \hat{h}_k}{\partial \hat{R}_j} \right] \times \frac{\partial \hat{R}_j}{\partial \hat{h}_j} \times \frac{\partial \hat{h}_j}{\partial w_{ijl}^{ci}}$$

$$\Delta w_{ijl}^{ci} = \in \left[\sum_k (c_k - R_k) \times R_k (1 - R_k) \times v_{jkl}^{ci} \right]$$

$$\times \hat{R}_k (1 - \hat{R}_k) \beta_i$$

$$\Delta w_{ijl}^{ci} = \in \left[\sum_k (c_k - \hat{R}_k) \times \hat{R}_k (1 - \hat{R}_k) \times v_{jkl}^{ci} \right]$$

$$\times R_j (1 - R_j) \beta_i$$

$$\Delta w_{ijl}^{ci} = \in \left[\sum_k \phi_k (v_{jkl}^{ci}) \right] \times \hat{R}_j (1 - \hat{R}_j) \beta_i$$

$$\Delta w_{ijl}^{ci} = \in \phi_j \beta_i \quad (10)$$

The mathematical form of final equation for input and hidden layer is given in equation 11.

$$w_{ijl}^{ci} = w_{ijl}^{ci} + \sigma_L (\Delta w_{ijl}^{ci}) \quad (11)$$

The mathematical form of final equation for output and hidden layer is given in equation 12.

$$v_{ijl}^{ci} = v_{ijl}^{ci} + \sigma_L (\Delta v_{ijl}^{ci}) \quad (12)$$

In equation 11 and equation 12 the σ_L represents the learning rate of proposed model.

F. AGGREGATION SERVER SIDE

Once the weights have been received on aggregation server, we need to preprocess the weights to avoid any conflict of dimension in the weights of contributors. There can be a conflict in the dimension of weights received from different contributors. First of all, we calculate the dimensions of weights using shape function. If the dimensions have conflict, then to avoid this conflict, we used zero padding technique. Suppose the highest dimension is (M x N) and

the lowest dimension is (K x L) then to equalize dimensions, zeros would be added to all the low dimensional weights. To create the zero-padding matrix, we used zeros function from TensorFlow. Once the dimensions are scaled next step on aggregation server is to aggregate the weights.

To aggregate the weights from equation 11 and 12 on server side, equation 13 has been used, lets dissect this equation. I_p represents the datapoints used by a contributor, and l represents the size of overall training data, where $\frac{l_p}{l}$ is scaling

factor and $p = 1, 2, 3, \dots, P$. The scaling factor is multiplied with weights of each contributor, where α represents weights v and w . After scaling the weights of each contributor, we sum up the scaled weights.

$$\mathcal{A}(\alpha^{ci}) = \sum_{p=1}^P \sum_{ijl} \frac{l_p}{l} F(\alpha^{ci}) \quad (13)$$

The basic work flow of the proposed model is shown in figure 2. This work flow diagram illustrates the working of each contributor and flow of data and parameters from initial stage to aggregation server. Each contributor in the system follows the same sequence and methodology.

III. SIMULATION RESULTS

To simulate the proposed model, we used Jupyter notebook, Python, Pandas, NumPy, Matplotlib, TensorFlow, Keras and scikit-learn (SKLearn). The system used for simulations has the specification of 16GB RAM, Intel Core i5-5300U CPU @ 2.30GHz, and 256GB solid-state drive (SSD).

Figure 3 shows global accuracy and global loss for 10 iterations and 3 contributors. It shows that global loss started from 0.08 and decreased gradually until it reached 0.05. Then the global loss drops further and touches 0.04. Later on, it keeps around 0.04. The global accuracy graph shows that initially, the model gave an accuracy of around 0.96, then it boosted to 0.98. Later on, it slightly increased above 0.98, but in most of the iterations, it is near to 0.98.

Figure 4 shows global accuracy and global loss for 30 iterations and 3 contributors. It shows that global loss started from 0.08 and decreased gradually until it reached 0.05. Then the global loss drops further and touches 0.04. Later on, it keeps around 0.04. The global accuracy graph shows that initially, the model gave an accuracy of around 0.96, then it boosted to 0.98. Later on, it slightly increased above 0.98, but in most of the iterations, it is around 0.98.

Figure 5 shows global accuracy and global loss for 50 iterations and 3 contributors. It shows that global loss started from 0.08 and decreased gradually until it reached 0.05. Then the global loss drops further and touches 0.03. But on average, it keeps around 0.04. The global accuracy graph shows that initially, the model gave an accuracy of around 0.96, then it boosted to 0.98. Later on, it slightly increased above 0.98, but in most of the iterations, it is around 0.98.

Figure 6 shows global accuracy and global loss for 10 iterations and five contributors. It shows that global loss started from 0.08 and decreased gradually until it reached 0.055. Then the global loss drops further and touches 0.045. Later

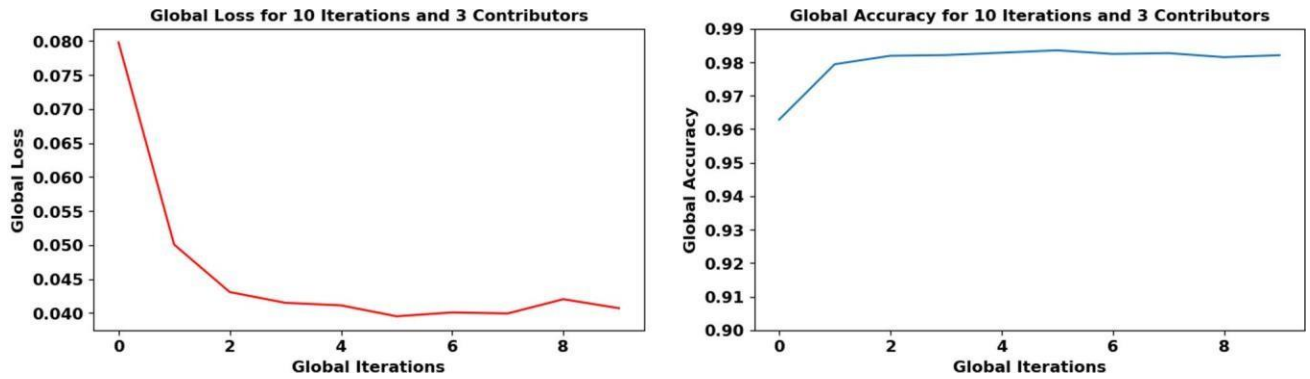


FIGURE 3. Global loss and accuracy, plot 1.

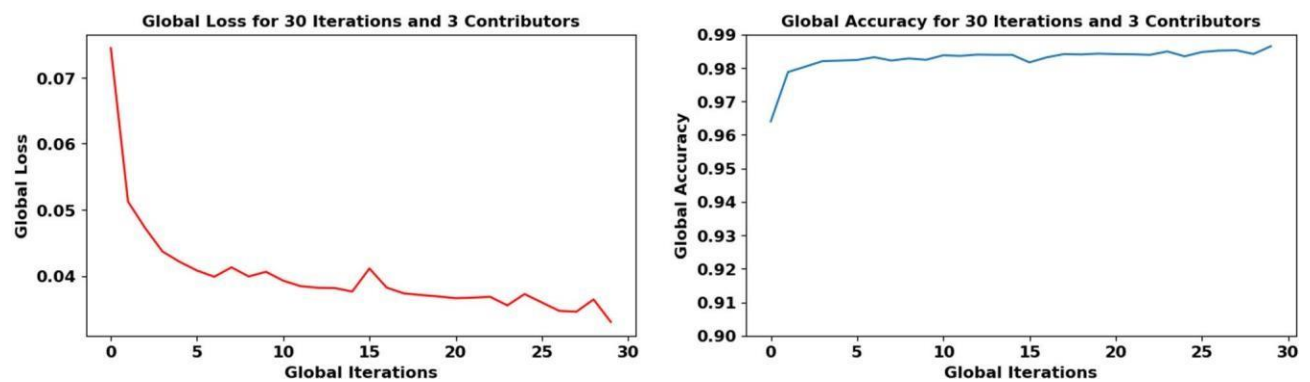


FIGURE 4. Global loss and accuracy, plot 2.

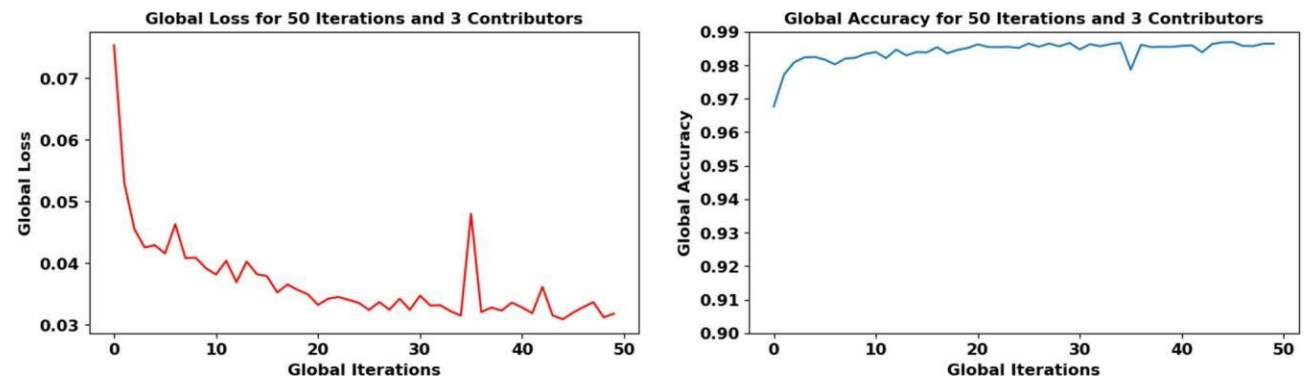


FIGURE 5. Global loss and accuracy, plot 3.

on, it keeps around 0.045. The global accuracy graph shows that initially, the model gave an accuracy of around 0.96, then it boosted to 0.98. Later on, it slightly increased above 0.98, but in most of the iterations, it stuck to 0.98.

Figure 7 shows global accuracy and global loss for 30 iterations and 5 contributors. It shows that global loss started from 0.085 and decreased gradually until it reached 0.06. Then the global loss drops further and touches 0.045. Later on, it keeps around 0.04. The global accuracy graph shows that initially,

the model gave an accuracy of around 0.96, then it boosted to 0.98. Later on, it slightly increased above 0.98, but in most of the iterations, it stuck to 0.98.

Figure 8 shows global accuracy and global loss for 50 iterations and 5 contributors. It shows that global loss started from 0.083 and decreased gradually until it reached 0.04. Later on, it drops further to 0.035, but on average, it keeps around 0.04. The global accuracy graph shows that initially, the model gave an accuracy of around 0.96, then it boosted to

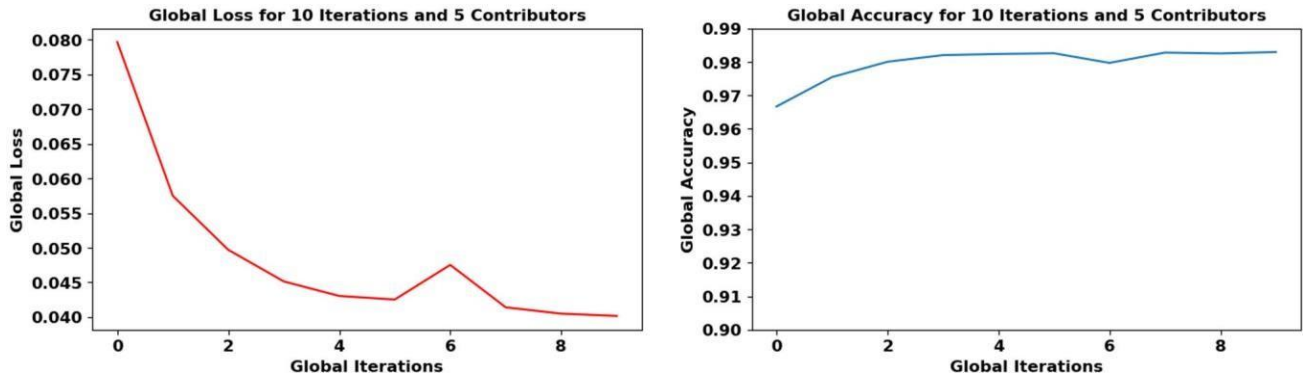


FIGURE 6. Global loss and accuracy, plot 4.

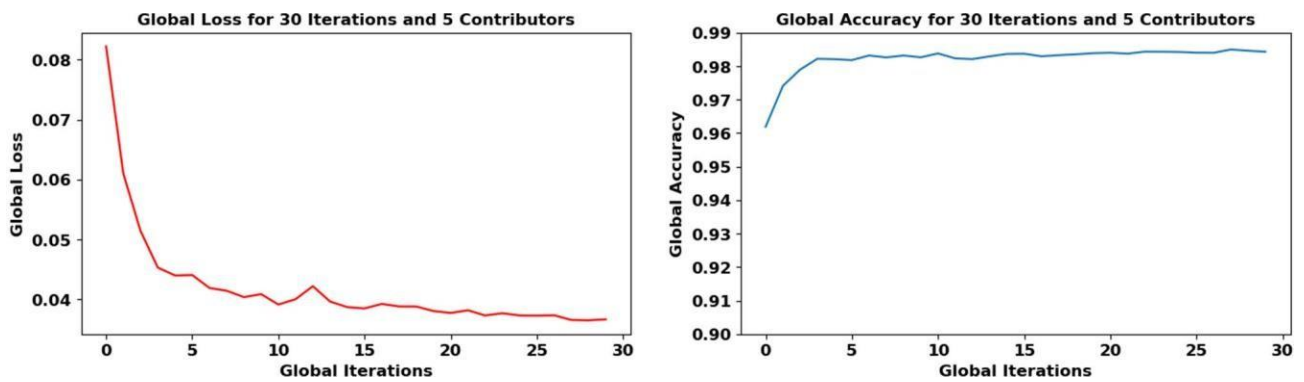


FIGURE 7. Global loss and accuracy, plot 5.

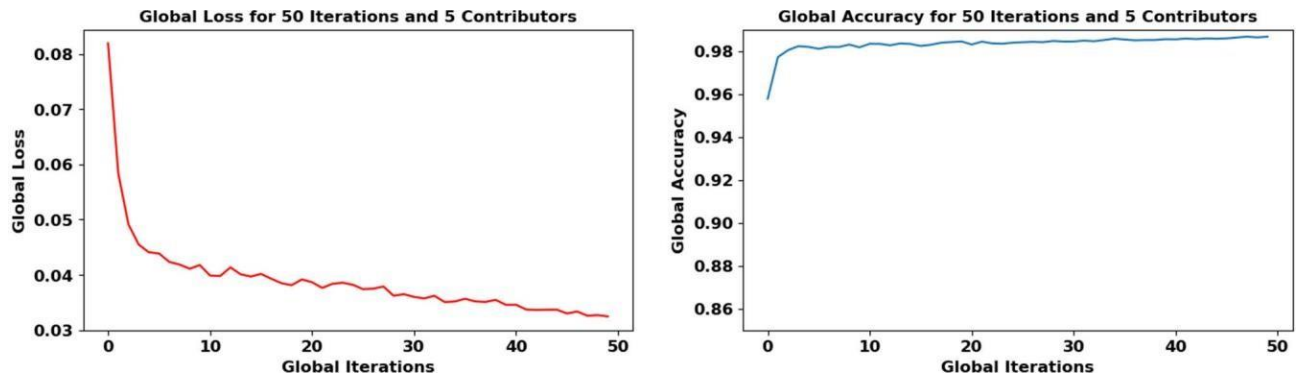


FIGURE 8. Global loss and accuracy, plot 6.

0.98. Later on, it slightly increased above 0.98, but in most of the iterations, it stuck to 0.98.

Figure 9 shows global accuracy and global loss for 10 iterations and 10 contributors. It shows that global loss started from 0.09 and decreased gradually until it reached 0.05. Then the global loss drops further and touches 0.04. Later on, it keeps around 0.04. The global accuracy graph shows that initially, the model gave an accuracy of around 0.96, then it

boosted to 0.98. Later on, it slightly increased above 0.98, but in most of the iterations, it stuck to 0.98.

Figure 10 shows global accuracy and global loss for 30 iterations and 10 contributors. It shows that global loss started from 0.09 and decreased gradually until it reached 0.05. Then the global loss drops further and touches 0.04. Later on, it keeps around 0.04. The global accuracy graph shows that initially, the model gave an accuracy of around 0.96, then it

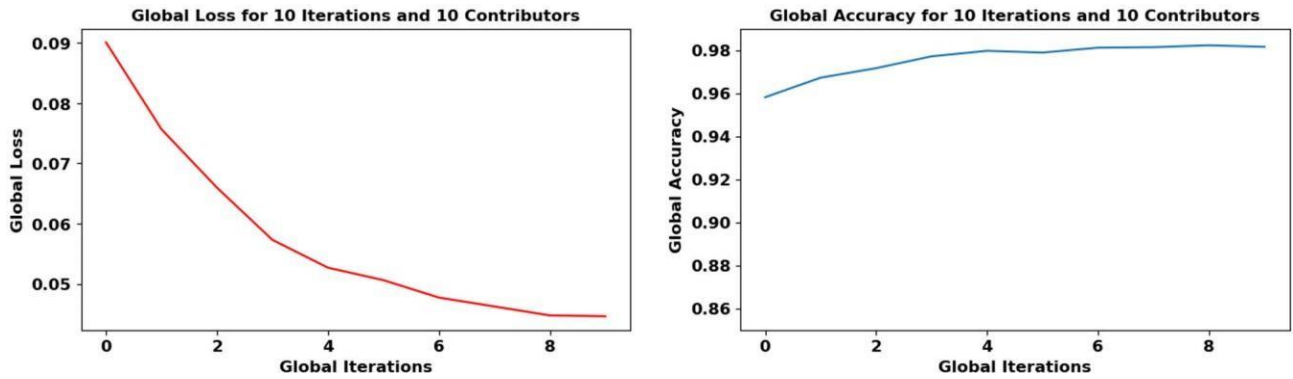


FIGURE 9. Global loss and accuracy, plot 7.

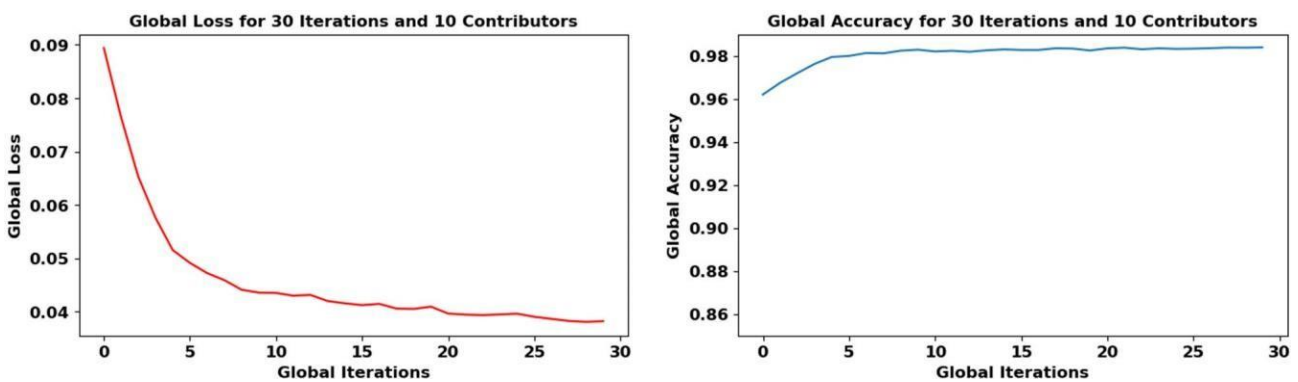


FIGURE 10. Global loss and accuracy, plot 8.

boosted to 0.98. Later on, it slightly increased above 0.98, but in most of the iterations, it stuck to 0.98.

Figure 11 shows global accuracy and global loss for 50 iterations and 10 contributors. It shows that global loss started from 0.089 and decreased gradually until it reached 0.05. Then the global loss drops further and touches 0.04. Later on, it keeps around 0.04. The global accuracy graph shows that initially, the model gave an accuracy of around 0.96, then it boosted to 0.98. Later on, it slightly increased above 0.98, but in most of the iterations, it maintained at 0.98.

Figure 12 shows global accuracy and global loss for 100 iterations and 10 contributors. It shows that global loss started from 0.094 and decreased gradually until it reached

0.05. Then the global loss drops further and touches 0.04.

Later on, it keeps around 0.04. The global accuracy graph shows that initially, the model gave an accuracy of around 0.95, then it boosted to 0.98. Later on, it slightly increased above 0.98, but in most of the iterations, it maintained at 0.98.

Table 2 summarizes the achieved simulation results. It shows change in contributors to 3, 5, and 10. It also shows change in global iterations to 10, 30, 50, and 100. Simulation results show that lowest accuracy obtained is 98.16%, and the highest is 98.65%. The global loss keeps around 0.04. The

lowest global loss obtained is 0.031, and the highest is 0.44. These simulation results show how effective the proposed model can be.

The evaluation of proposed model is investigated using several evaluation metrics including accuracy score, precision score, recall, and f1 score. The outcomes of these metrics help to estimate performance of model. These metrics give a score of around 0.98.6%, which meet the simulation results we discussed earlier. These results verifies that the performance and accuracy of the proposed model is excellent. Table 3 shows the score of evaluation metrics we used to evaluate the proposed model.

To prove the novelty and effectiveness of proposed model, we also performed the centralized IDS (CIDS) model training and evaluation using the Edge-IIoTset dataset. We trained the 5 contributors independently using their local datasets. The experimental results are given in Table 4. We found that there is a significant difference between the accuracy and performance of centralized IDS and FL based IDS. The accuracy and performance of centralized IDS is lower than FL based IDS. The highest accuracy achieved by an independent centralized contributor is 78.46% whereas the highest accuracy achieved by FL based IDS is 98.65%.

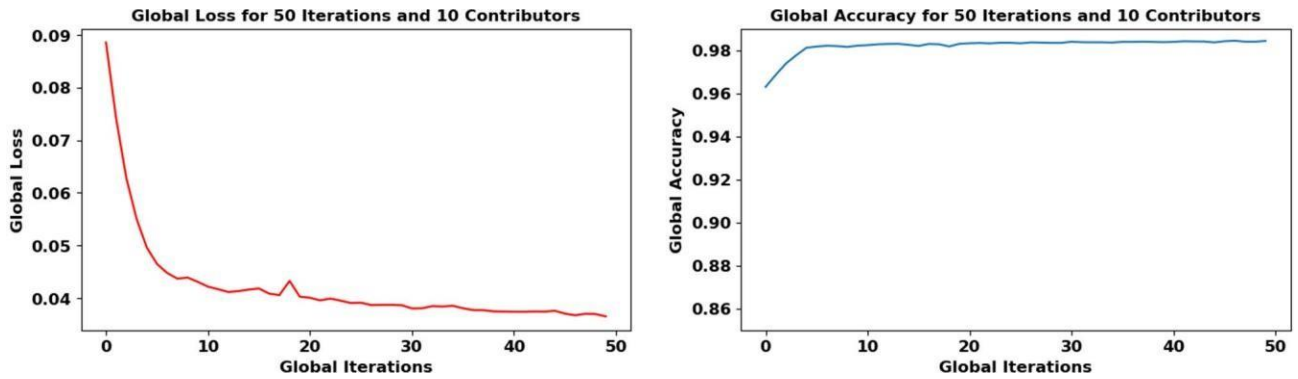


FIGURE 11. Global loss and accuracy, plot 9.

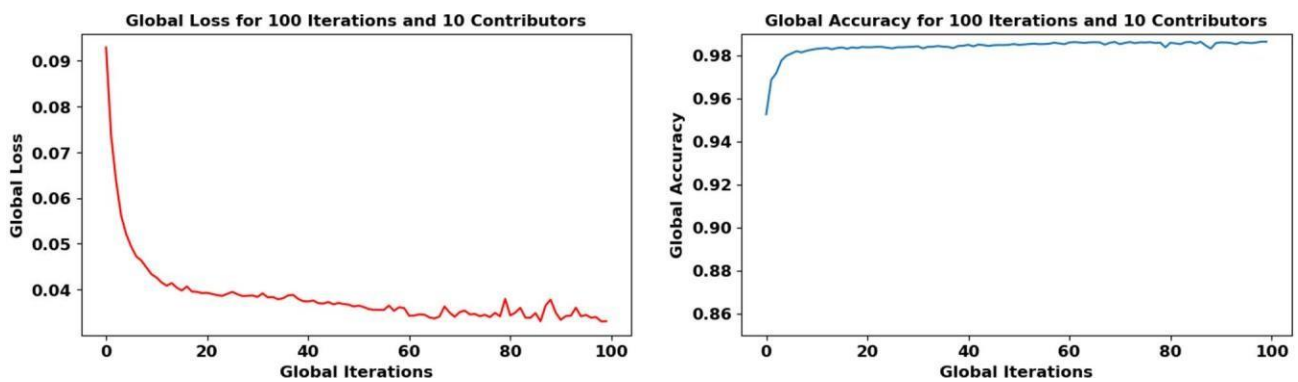


FIGURE 12. Global loss and accuracy, plot 10.

TABLE 3. Scores of evaluation metrics.

Sr. No.	Evaluation Metric	Score
1	Accuracy Score	0.9865
2	Precision Score (micro)	0.9863
3	Precision Score (weighted)	0.9864
4	Recall Score (micro)	0.9863
5	Recall Score (weighted)	0.9863
6	f1 Score (micro)	0.9863
7	f1 Score (weighted)	0.9864

TABLE 4. Results of centralized IDS.

Sr. No.	Centralized IDS	Accuracy Score
1	CIDS1	0.7845
2	CIDS2	0.7843
3	CIDS3	0.7846
4	CIDS4	0.7836
5	CIDS5	0.7744

IV. DISCUSSION

The proposed model can preserve data privacy of training data using FL. Further, the proposed model solves the problem of

training data unavailability due to privacy concerns. We used ANN as a baseline model in proposed model. We tested the proposed model using the latest and most realistic cybersecurity dataset [14]. We evaluated the proposed model using seven evaluation metrics. These evaluation metrics helped to assess the proposed model in detail. The proposed model has been tested using different global iterations and contributors. According to the evaluation results of the proposed model, we obtained 98.65% accuracy.

In the literature review considered several research articles. Let's discuss their contributions. ElSayed et al. proposed a hybrid deep learning approach based on CNN to classify normal or attack classes [20] in SDN. They used precision, recall, and F1-score metrics to assess the model. Ahmed et al. proposed an intrusion detection algorithm named deep active learning [23]. They used F1-score metrics for evaluation. Ravi et al. proposed a deep learning approach for intrusion detection in SDN-IoT networks [21]. They used precision, recall, and F1-score metrics for evaluation. Phan and Bauschert proposed a reinforcement learning based solution for intrusion detection in SDN [18]. They proposed the solution to detect Dos attacks. Zhu et al. proposed a Resnet based solution for intrusion detection in SDN [17]. Yang et al. proposed autoencoder based intrusion detections system for

TABLE 5. Comparison results of proposed model with literature.

Studies	Machine Learning Techniques	Privacy Preservation	Accuracy	Detection Classes
Zhu et al. [17]	Resnet	No	93.52%	2
Phan et al. [18]	Q-Networks	No	85%	2
ElSayed et al. [20]	CNN	No	97%	2
Yang et al. [22]	Ensemble Autoencoder	No	97%	2
Ahmed et al. [23]	Active Deep Learning	No	96%	2
Xu et al. [51]	KNN	No	97.6%	2
Ahmad et al. [38]	FL	Yes	95.41%	6
Jie et al. [40]	FL	Yes	98%	5
Zakaria et al. [44]	FL	Yes	89%	5
Proposed Model	FL	Yes	98.65%	11

SDN [22]. They used unsupervised learning technique in their proposed model.

We discussed the findings and results of the proposed model and the results and techniques of earlier studies. Let's compare them with the proposed model. All of them used centralized machine learning techniques. We used FL that is a distributed learning technique. Further the proposed model obtained 98.65% accuracy whereas earlier models obtained accuracy around 85% to 97%. They used 3 to 4 evaluation metrics for evaluation. We used seven evaluation metrics for the evaluation of the proposed model. Their proposed models do not address the problem of data privacy preservation, and the problem of unavailability of training data due to privacy concerns. The proposed model addresses these problems. They used old datasets. None of them used the latest dataset we used. The cited research articles proposed IDS which can classify 1 or 2 classes only. The proposed model can classify 11 classes. This detailed comparison shows how the proposed model is better than earlier studies.

FL is a collaborative machine learning technique which uses weights or parameters for the training of global model [8], [47], [49], [50]. Every organization and individual being is concerned for the privacy of data. They do not willing to share their data specially when it comes to the data of network traffic. In FL different organizations can share the parameters of their model training instead of training data. IDS based on centralized machine learning techniques suffer from data privacy preservation and unavailability of training data due to privacy problem. FL provides a fascinating solution against these problems. So, in IDS for SDN, FL is more effective approach then centralized machine learning techniques.

Simulation results show that varying the number of contributors does not affect the accuracy and performance of proposed model. More global iterations can give more precise results but we find that 50 to 100 global iterations are enough although they can be varied depending upon the dataset. More global iterations require more computation power so

the optimal number of global iterations play a crucial role in the training of proposed system.

A. IMPLEMENTATION CHALLENGES AND SOLUTIONS

SDN networks can involve a diverse range of devices with varying processing power, memory limitations, and communication bandwidths. This heterogeneity can impact the efficiency of the federated learning process. Carefully selecting the training devices with required computational power can optimize the learning process. FL based IDS involves exchanging model updates between devices and the central server. Excessive communication can strain network resources and increase latency. A highly efficient communication network can reduce the communication overhead and latency. Otherwise, we can Implement dynamic scheduling algorithms that consider factors like device load and communication bandwidth to optimize model update frequency.

B. PRACTICAL IMPLICATIONS AND APPLICATIONS

SDNs are widely used in telecommunications for network management, virtualization and configurations. The proposed IDS can be used to detect and prevent network intrusions in telecommunications networks. SDNs are used in cloud computing for network virtualization and resource allocation. The proposed IDS can be used to detect and prevent network intrusions in cloud computing environments. Networks implemented and used in IoT are prone to attacks. SDNs are used in IoT for network management. The proposed model can be used to detect and prevent network intrusions in IoT networks.

We have emphasized the importance of data privacy in SDNs and how FL can help preserve data privacy by enabling local data processing and training. The proposed IDS is designed to be scalable, making it suitable for large-scale SDNs. We have discussed how FL enables the system to learn from distributed data sources, reducing the need for centralized data storage and processing. FL reduces network latency by enabling local data processing and analysis, reducing the need for data transfer across the network and devices.

V. CONCLUSION

In this research article we have demonstrated the promising potential of FL in the context of intrusion detection within SDNs. We have validated the proposed model using a latest cyber security dataset and compared the proposed model with state of art existing studies. Achieving a remarkable accuracy rate of 98.65%, findings of this research highlight the effectiveness of collaborative machine learning approach in enhancing network security. However, achieving such high accuracy rates requires careful model selection, hyperparameter tuning and data processing. By distributing the training process across multiple network nodes while preserving data privacy, FL permits real-time threat detection and adaptation to evolving attacks. This not only enhances the overall security of SDNs but also minimizes the risk of data exposure and solves the problem of training data privacy concerns. The proposed model is a multi-class classification approach. As the landscape of cyber threats continues to evolve, this research work underscores the importance of leveraging cutting-edge technologies like FL. The proposed model is an innovative IDS to mitigate the flaws and challenges of traditional and centralized IDS.

VI. FUTURE DIRECTIONS

In the future, we will explore more datasets to include more important classes to the training data and to compare the performance of proposed model. This will increase the scope and effectiveness of proposed model and ensure the in-depth validation.

REFERENCES

- [1] S. Lee, S. Woo, J. Kim, J. Nam, V. Yegneswaran, P. Porras, and S. Shin, "A framework for policy inconsistency detection in software-defined networks," *IEEE/ACM Trans. Netw.*, vol. 30, no. 3, pp. 1410–1423, Jun. 2022, doi: [10.1109/TNET.2022.3140824](https://doi.org/10.1109/TNET.2022.3140824).
- [2] K. K. Karmakar, V. Varadharajan, and U. Tupakula, "Mitigating attacks in software defined networks," *Cluster Comput.*, vol. 22, no. 4, pp. 1143–1157, Dec. 2019.
- [3] M. Priyadarsini and P. Bera, "Software defined networking architecture, traffic management, security, and placement: A survey," *Comput. Netw.*, vol. 192, Jun. 2021, Art. no. 108047, doi: [10.1016/j.comnet.2021.108047](https://doi.org/10.1016/j.comnet.2021.108047).
- [4] W. Rafique, L. Qi, I. Yaqoob, M. Imran, R. U. Rasool, and W. Dou, "Complementing IoT services through software defined networking and edge computing: A comprehensive survey," *IEEE Commun. Surveys Tuts.*, vol. 22, no. 3, pp. 1761–1804, 3rd Quart., 2020.
- [5] M. F. Hyder and M. A. Ismail, "Securing control and data planes from reconnaissance attacks using distributed shadow controllers, reactive and proactive approaches," *IEEE Access*, vol. 9, pp. 21881–21894, 2021.
- [6] I. H. Sarker, A. S. M. Kayes, S. Badsha, H. Alqahtani, P. Watters, and A. Ng, "Cybersecurity data science: An overview from machine learning perspective," *J. Big Data*, vol. 7, no. 1, pp. 1–29, Dec. 2020.
- [7] G. Apruzzese, P. Laskov, E. M. de Oca, W. Mallouli, L. B. Rapa, A. V. Grammatopoulos, and F. Di Franco, "The role of machine learning in cybersecurity," *Digit. Threats, Res. Pract.*, vol. 4, no. 1, pp. 1–38, 2022.
- [8] C. Zhang, Y. Xie, H. Bai, B. Yu, W. Li, and Y. Gao, "A survey on federated learning," *Knowl.-Based Syst.*, vol. 216, Jun. 2021, Art. no. 106775.
- [9] T. Li, A. K. Sahu, A. Talwalkar, and V. Smith, "Federated learning: Challenges, methods, and future directions," *IEEE Signal Process. Mag.*, vol. 37, no. 3, pp. 50–60, May 2020.
- [10] M. Chen, U. Challita, W. Saad, C. Yin, and M. Debbah, "Artificial neural networks-based machine learning for wireless networks: A tutorial," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 4, pp. 3039–3071, 4th Quart., 2019.
- [11] Q. Li, Z. Wen, Z. Wu, S. Hu, N. Wang, Y. Li, X. Liu, and B. He, "A survey on federated learning systems: Vision, hype and reality for data privacy and protection," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 4, pp. 3347–3366, Apr. 2023.
- [12] J. Lansky, S. Ali, M. Mohammadi, M. K. Majeed, S. H. T. Karim, S. Rashidi, M. Hosseinzadeh, and A. M. Rahmani, "Deep learning-based intrusion detection systems: A systematic review," *IEEE Access*, vol. 9, pp. 101574–101599, 2021.
- [13] A. Khraisat, I. Gondal, P. Vamplew, and J. Kamruzzaman, "Survey of intrusion detection systems: Techniques, datasets and challenges," *Cyber-security*, vol. 2, no. 1, pp. 1–22, Dec. 2019.
- [14] M. A. Ferrag, O. Friha, D. Hamouda, L. Maglaras, and H. Janicke, "Edge-IIoTset: A new comprehensive realistic cyber security dataset of IoT and IIoT applications for centralized and federated learning," *IEEE Access*, vol. 10, pp. 40281–40306, 2022.
- [15] M. P. Novaes, L. F. Carvalho, J. Lloret, and M. L. Proença, "Long short-term memory and fuzzy logic for anomaly detection and mitigation in software-defined network environment," *IEEE Access*, vol. 8, pp. 83765–83781, 2020.
- [16] S. Haider, A. Akhunzada, I. Mustafa, T. B. Patel, A. Fernandez, K. R. Choo, and J. Iqbal, "A deep CNN ensemble framework for efficient DDoS attack detection in software defined networks," *IEEE Access*, vol. 8, pp. 53972–53983, 2020.
- [17] Z. Zhu, W. Zhai, H. Liu, J. Geng, M. Zhou, C. Ji, and G. Jia, "Juggler-ResNet: A flexible and high-speed ResNet optimization method for intrusion detection system in software-defined industrial networks," *IEEE Trans. Ind. Informat.*, vol. 18, no. 6, pp. 4224–4233, Jun. 2022.
- [18] T. V. Phan and T. Bauschert, "DeepAir: Deep reinforcement learning for adaptive intrusion response in software-defined networks," *IEEE Trans. Netw. Service Manage.*, vol. 19, no. 3, pp. 2207–2218, Sep. 2022, doi: [10.1109/TNSM.2022.3158468](https://doi.org/10.1109/TNSM.2022.3158468).
- [19] K. S. Sahoo, B. K. Tripathy, K. Naik, S. Ramasubbareddy, B. Balusamy, M. Khari, and D. Burgos, "An evolutionary SVM model for DDOS attack detection in software defined networks," *IEEE Access*, vol. 8, pp. 132502–132513, 2020.
- [20] M. S. ElSayed, N.-A. Le-Khac, M. A. Albahar, and A. Jurcut, "A novel hybrid model for intrusion detection systems in SDNs based on CNN and a new regularization technique," *J. Netw. Comput. Appl.*, vol. 191, Oct. 2021, Art. no. 103160.
- [21] V. Ravi, R. Chaganti, and M. Alazab, "Deep learning feature fusion approach for an intrusion detection system in SDN-based IoT networks," *IEEE Internet Things Mag.*, vol. 5, no. 2, pp. 24–29, Jun. 2022.
- [22] L. Yang, Y. Song, S. Gao, A. Hu, and B. Xiao, "Griffin: Real-time network intrusion detection system via ensemble of autoencoder in SDN," *IEEE Trans. Netw. Service Manage.*, vol. 19, no. 3, pp. 2269–2281, Sep. 2022.
- [23] U. Ahmed, J. C. Lin, G. Srivastava, U. Yun, and A. K. Singh, "Deep active learning intrusion detection and load balancing in software-defined vehicular networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 1, pp. 953–961, Jan. 2023.
- [24] S. Prathibha, J. Bino, Md. T. Ahammed, C. Das, S. R. Oion, S. Ghosh, and M. Afroj, "Detection methods for software defined networking intrusions (SDN)," in *Proc. Int. Conf. Adv. Comput., Commun. Appl. Informat. (ACCAI)*, Jan. 2022, pp. 1–6.
- [25] Y. Cui, Q. Qian, C. Guo, G. Shen, Y. Tian, H. Xing, and L. Yan, "Towards DDoS detection mechanisms in software-defined networking," *J. Netw. Comput. Appl.*, vol. 190, Sep. 2021, Art. no. 103156.
- [26] R. U. Rasool, U. Ashraf, K. Ahmed, H. Wang, W. Rafique, and Z. Anwar, "Cyberpulse: A machine learning based link flooding attack mitigation system for software defined networks," *IEEE Access*, vol. 7, pp. 34885–34899, 2019.
- [27] I. Feki, S. Ammar, Y. Kessentini, and K. Muhammad, "Federated learning for COVID-19 screening from chest X-ray images," *Appl. Soft Comput.*, vol. 106, Jul. 2021, Art. no. 107330.
- [28] J. Li, Y. Meng, L. Ma, S. Du, H. Zhu, Q. Pei, and X. Shen, "A federated learning based privacy-preserving smart healthcare system," *IEEE Trans. Ind. Informat.*, vol. 18, no. 3, pp. 2021–2031, Mar. 2022.
- [29] B. Han, R. H. Jhaveri, H. Wang, D. Qiao, and J. Du, "Application of robust zero-watermarking scheme based on federated learning for securing the healthcare data," *IEEE J. Biomed. Health Informat.*, vol. 27, no. 2, pp. 804–813, Feb. 2023.
- [30] J. Fan, X. Wang, Y. Guo, X. Hu, and B. Hu, "Federated learning driven secure Internet of Medical Things," *IEEE Wireless Commun.*, vol. 29, no. 2, pp. 68–75, Apr. 2022.

- [31] K. Guo, T. Chen, S. Ren, N. Li, M. Hu, and J. Kang, "Federated learning empowered real-time medical data processing method for smart healthcare," *IEEE/ACM Trans. Comput. Biol. Bioinf.*, early access, Jun. 2022.
- [32] H. Elayan, M. Aloqaily, and M. Guizani, "Sustainability of healthcare data analysis IoT-based systems using deep federated learning," *IEEE Internet Things J.*, vol. 9, no. 10, pp. 7338–7346, May 2022.
- [33] M. Driss, I. Almomani, Z. Huma, and J. Ahmad, "A federated learning framework for cyberattack detection in vehicular sensor networks," *Complex Intell. Syst.*, vol. 8, no. 5, pp. 4221–4235, Oct. 2022.
- [34] J. Li, X. Tong, J. Liu, and L. Cheng, "An efficient federated learning system for network intrusion detection," *IEEE Syst. J.*, vol. 17, no. 2, pp. 2455–2464, Jan. 2023.
- [35] X. Huang, J. Liu, Y. Lai, B. Mao, and H. Lyu, "EEFED: Personalized federated learning of execution&evaluation dual network for CPS intrusion detection," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 41–56, 2023.
- [36] R. Zhao, Y. Wang, Z. Xue, T. Ohtsuki, B. Adebisi, and G. Gui, "Semisupervised federated-learning-based intrusion detection method for Internet of Things," *IEEE Internet Things J.*, vol. 10, no. 10, pp. 8645–8657, May 2023.
- [37] H. Liang, D. Liu, X. Zeng, and C. Ye, "An intrusion detection method for advanced metering infrastructure system based on federated learning," *J. Mod. Power Syst. Clean Energy*, vol. 11, no. 3, pp. 927–937, May 2023, doi: [10.35833/MPCE.2021.000279](https://doi.org/10.35833/MPCE.2021.000279).
- [38] A. Zainudin, R. Akter, D.-S. Kim, and J.-M. Lee, "Federated learning inspired low-complexity intrusion detection and classification technique for SDN-based industrial CPS," *IEEE Trans. Netw. Service Manage.*, vol. 20, no. 3, pp. 2442–2459, Sep. 2023.
- [39] H. Babbar and S. Rani, "FRHIDS: Federated learning recommender hybrid intrusion detection system model in software-defined networking for consumer devices," *IEEE Trans. Consum. Electron.*, vol. 70, no. 1, pp. 2492–2499, Feb. 2024.
- [40] J. Cui, H. Sun, H. Zhong, J. Zhang, L. Wei, I. Bolodurina, and D. He, "Collaborative intrusion detection system for SDVN: A fairness federated deep learning approach," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 9, pp. 2512–2528, Sep. 2023.
- [41] Z. A. E. Houda, H. Moudoud, and B. Brik, "Federated deep reinforcement learning for efficient jamming attack mitigation in O-RAN," *IEEE Trans. Veh. Technol.*, early access, Jan. 2024.
- [42] Z. A. E. Houda, H. Moudoud, B. Brik, and L. Khoukhi, "Blockchain-enabled federated learning for enhanced collaborative intrusion detection in vehicular edge computing," *IEEE Trans. Intell. Transp. Syst.*, early access, Jan. 2024.
- [43] Z. Abou El Houda, H. Moudoud, B. Brik, and L. Khoukhi, "Securing federated learning through blockchain and explainable AI for robust intrusion detection in IoT networks," in *Proc. IEEE Conf. Comput. Commun. Workshops*, May 2023, pp. 1–6.
- [44] Z. A. E. Houda, A. S. Hafid, and L. Khoukhi, "MiTFed: A privacy preserving collaborative network attack mitigation framework based on federated learning using SDN and blockchain," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 4, pp. 1985–2001, Jan. 2023.
- [45] Z. A. E. Houda, B. Brik, A. Ksentini, L. Khoukhi, and M. Guizani, "When federated learning meets game theory: A cooperative framework to secure IIoT applications on edge computing," *IEEE Trans. Ind. Informat.*, vol. 18, no. 11, pp. 7988–7997, Nov. 2022.
- [46] S. M. S. Bukhari, M. H. Zafar, M. A. Houran, S. K. R. Moosavi, M. Mansoor, M. Muaz, and F. Sanfilippo, "Secure and privacy-preserving intrusion detection in wireless sensor networks: Federated learning with SCNN-bi-LSTM for enhanced reliability," *Ad Hoc Netw.*, vol. 155, Mar. 2024, Art. no. 103407.
- [47] A. Qayyum, K. Ahmad, M. A. Ahsan, A. Al-Fuqaha, and J. Qadir, "Collaborative federated learning for healthcare: Multi-modal COVID-19 diagnosis at the edge," *IEEE Open J. Comput. Soc.*, vol. 3, pp. 172–184, 2022.
- [48] T. Haibo, L. Maonan, and R. Shuangyin, "ESE: Efficient security enhancement method for the secure aggregation protocol in federated learning," *Chin. J. Electron.*, vol. 32, no. 3, pp. 542–555, May 2023.
- [49] B. Liu, M. Ding, S. Shaham, W. Rahayu, F. Farokhi, and Z. Lin, "When machine learning meets privacy: A survey and outlook," *ACM Comput. Surv.*, vol. 54, no. 2, pp. 1–36, Mar. 2022.
- [50] J. Xu, B. S. Glicksberg, C. Su, P. Walker, J. Bian, and F. Wang, "Federated learning for healthcare informatics," *J. Healthcare Informat. Res.*, vol. 5, no. 1, pp. 1–19, Mar. 2021.

- [51] Y. Xu, H. Sun, F. Xiang, and Z. Sun, "Efficient DDoS detection based on K-FKNN in software defined networks," *IEEE Access*, vol. 7, pp. 160536–160545, 2019.



MUBASHAR RAZA received the B.Sc. degree in computer systems engineering from The Islamia University of Bahawalpur, Pakistan, in 2019, and the M.S. degree in computer science from Riphah International University, Lahore Campus, Pakistan, in 2023.

Since 2023, he has been a Lecturer in computing with Riphah International University, Sahiwal Campus, Pakistan. His research interests include machine learning, computer networks, network security, computer vision, natural language processing, robotics, and information security.



MUHAMMAD JASIM SAEED received the M.S. degree in computer science from Liverpool John Moores University, U.K., and the Ph.D. degree in computer communications and networks from Manchester Metropolitan University, U.K.

Currently, he holds the position of an Assistant Professor and the Head of the Department of Computing, Riphah International University, Lahore Campus, Pakistan.



MUHAMMAD BILAL RIAZ received the Ph.D. degree in applied science in Pakistan.

He is currently a Senior Researcher with the VSB—Technical University of Ostrava, Czech Republic. He has more than 200 publications in highly reputed journals. His research interests include computational mathematics, mathematics physics, fluid dynamics, differential equations, dynamical systems, chaos theory, fractional calculus, and its applications in sciences.



MUHAMMAD AWAIS SATTAR received the B.Sc. degree in electrical (computer) engineering from COMSATS University, Lahore, Pakistan, in 2014, the M.Sc. degree in electrical engineering from Rochester Institute of Technology, Dubai, United Arab Emirates, in 2017, and the Ph.D. degree in technical computer science and telecommunication from Lodz University of Technology, Łódź, Poland, in 2022.

He is currently an Assistant Professor of computing with Riphah International University, Lahore Campus, Pakistan. His research interests include machine learning, computer vision, and process tomography.

...

Sudha M

report_check

 journal

Document Details

Submission ID

trn:oid::3618:137627461

Submission Date

May 4, 2026, 5:07 PM GMT+5:30

Download Date

May 4, 2026, 5:09 PM GMT+5:30

File Name

report_check.docx

File Size

51.4 KB

15 Pages**4,879 Words****28,529 Characters**

11% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

-  **58 Not Cited or Quoted** 11%
Matches with neither in-text citation nor quotation marks
-  **2 Missing Quotations** 0%
Matches that are still very similar to source material
-  **0 Missing Citation** 0%
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted** 0%
Matches with in-text citation present, but no quotation marks

Top Sources

- 6%  Internet sources
- 8%  Publications
- 7%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- 58** Not Cited or Quoted 11%
Matches with neither in-text citation nor quotation marks
- 2** Missing Quotations 0%
Matches that are still very similar to source material
- 0** Missing Citation 0%
Matches that have quotation marks, but no in-text citation
- 0** Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

- 6% Internet sources
- 8% Publications
- 7% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Internet		
www.mdpi.com			2%
2	Publication		
Sonali Dhananjay Patil, Rajesh Ingle, Amar Buchade, Vidy Potdar. "Decentralized ...			<1%
3	Publication		
Jaskaran Singh, Meenu Gupta, Rakesh Kumar. "Smart Technologies and Intelligen...			<1%
4	Student papers		
University of Birmingham on 2024-09-15			<1%
5	Publication		
Cheng Wang, Congjun Rao, Fuyan Hu, Xinping Xiao, Mark Goh. "Risk assessment ...			<1%
6	Student papers		
University of Alabama on 2016-03-28			<1%
7	Publication		
Arvind Dagur, Karan Singh, Pawan Singh Mehra, Dharendra Kumar Shukla. "Artific...			<1%
8	Student papers		
University of Hertfordshire on 2024-12-29			<1%
9	Publication		
Swati Sah, Rejwan Bin Sulaiman, Aditya Dayal Tyagi. "Federated Learning in Finan...			<1%
10	Internet		
assets-eu.researchsquare.com			<1%

11	Internet	ijetrm.com	<1%
12	Internet	thesesjournal.com	<1%
13	Student papers	National College of Ireland on 2024-12-11	<1%
14	Publication	Soufiane Haddout, Tonni Agustiono Kurniawan. "Handbook of Artificial Intelligen...	<1%
15	Internet	www.emerald.com	<1%
16	Publication	Harvinder Singh, Priyanka Kaushal, Sarabjeet Kaur. "Advanced Computing and AI...	<1%
17	Student papers	Kingston University on 2023-09-18	<1%
18	Student papers	SASTRA University on 2020-12-23	<1%
19	Internet	dspace.daffodilvarsity.edu.bd:8080	<1%
20	Internet	journals.ametsoc.org	<1%
21	Internet	nebula.wsimg.com	<1%
22	Internet	oasis-lmc.org	<1%
23	Publication	Kotoklo, Agbessi. "Enhancing Cyber Threat Detection on the Internet of Healthca...	<1%
24	Student papers	University of Surrey on 2026-03-13	<1%

25	Student papers	Corvinus University of Budapest on 2024-11-22	<1%
26	Publication	Farkhari, Hamed. "AI Based Cybersecurity Enhancement in 5G Networks.", ISCTE -...	<1%
27	Student papers	Moodle2025 on 2026-05-01	<1%
28	Student papers	Moodle2025 on 2026-05-02	<1%
29	Student papers	University of Westminster on 2026-04-17	<1%
30	Publication	Xin Qi, Tao Xu, Chengrun Dang, Zhuang Qi, Lei Meng, Han Yu. "Federated learnin...	<1%
31	Internet	docplayer.net	<1%
32	Internet	journal.yrpioku.com	<1%
33	Internet	norma.ncirl.ie	<1%
34	Internet	www.jist.ir	<1%
35	Publication	Hasan, H. Mahedi. "Analysis of Model Aggregation Techniques in Federated Learn...	<1%
36	Publication	Prasant Kumar Rout, Binita Nanda, Sunil Kumar Dhal. "A BERT-Based Framework ...	<1%
37	Student papers	University of Ulster on 2026-04-13	<1%
38	Publication	Utku Kose, Omer Deperlioglu, D. Jude Hemanth. "Deep Learning for Biomedical A...	<1%

ABSTRACT

The increasing complexity of cyber threats necessitates robust and privacy-protective intrusion detection systems (IDS). Network traffic logs are the fundamental input to train an efficient IDS. Unfortunately, these network traffic logs include sensitive information related to users' credentials, financial transactions, and/or company/organization communications; therefore, collecting and processing this type of information in a centralized manner is illegal due to regulatory bodies such as GDPR or HIPAA, and operational risks associated with a single point-of-failure. In contrast, Federated Learning (FL) provides a solution that allows multiple organizations to collaborate in developing AI-based models without transferring the underlying raw data from one organization to another. As part of this work, we provide an extensive comparative evaluation of five deep learning architectures – Multilayer Perceptron (MLP), Long Short-Term Memory (LSTM), Convolutional Neural Network (CNN), Deep Neural Network (DNN), and hybrid CNN + LSTM model – within two distinct frameworks (centralized vs. federated learning) using the widely-used NSL-KDD benchmark dataset which includes 148,520 samples. Additionally, federated learning experiments were conducted with five clients distributed across 20 communication rounds utilizing the FedProx aggregation method. Our results show that federated learning surpassed centralized learning for all architecture types and performance metrics. Specifically, federated LSTMs achieved the highest accuracy rate (97.55%), as well as the highest precision rate (98.37%), compared to all baseline values. Furthermore, our per-class analysis shows high quality attack detection for DoS (F1 = 99.82%) and Probe (F1 = 97.95%) attacks, whereas the ability to identify rare events (i.e., R2L and U2R minor classes) remains a challenge common to the community. Comparisons against six existing state-of-the-art approaches support the competitive nature of our proposed framework. Overall, the use of FL represents a technically superior, privacy- friendly, and deployable framework for implementing distributed IDS.

1.1 INTRODUCTION

We have seen the rapid expansion of our attack surface via cloud based services and Internet connected devices. As a result, we have the need for better tools to defend against these threats. One tool that provides us with some insight to this issue is Intrusion Detection Systems (IDS). IDS systems provide real-time identification of abnormal behavior or malicious activity occurring on the network. Although there have been many advancements in IDS technology, classical IDS technology does not support new or "zero-day" attacks. Therefore, the use of machine learning (ML) and deep learning (DL) technologies have emerged. These technologies allow IDS to learn how to detect anomalies through data [1]. There is however a major drawback when it comes to applying IDS using ML/DL technologies in real world distributed environments. In order for these IDS systems to be effective they need to be trained on large amounts of historical data. However, one problem that occurs is that the data required to train them is extremely sensitive and therefore cannot be shared across organizational boundaries.

For example, network logs typically include user names, session tokens, financial transactions, personal communications and internal architecture information. When you collect data like this for centralized model training there are three types of risk associated with doing so. Those risks are (i) violation of laws and regulations regarding data protection such as GDPR, PDPA and HIPAA. (ii) creating a single point of failure in terms of security because once organizations share their log data then the organization that stores that log data will become the most attractive target for an attacker. (iii) model inversion attacks where an attacker can determine sensitive training data from the gradients of the model that was used to classify their input [8]. Therefore, in addition to what would be technologically desirable (to have global visibility into threats across all participating networks), we now have significant legal and operational hurdles to overcome before we can deploy a privacy preserving IDS — thus privacy preservation is no longer optional, but rather necessary to make privacy preserving IDS deployable. Federated Learning (FL) addresses this privacy concern. Instead of sending the raw data to a central server, each client trains a local model on its own data and sends the model updates (gradients/weight differentials) back to a central server. At the central server, the model updates are combined (aggregated) to create a global model. By doing this, the clients do not expose their raw data to other parties [2]. FL has shown great promise in various domains including health care [9], mobile prediction [10], and detecting financial fraud [11]; however, despite being widely applicable to network intrusion detection, the application of FL to IDS is still largely unexplored due to the potential degradation of convergence caused by the non-identical and independent distribution (non-IID) of traffic patterns across clients [6, 7].

1.2 PROBLEM DEFINITION

The need to secure networks from attackers has become urgent because of the rapid rise in numbers of Internet-based services and networked systems. However, a traditional intrusion detection system - using a centralized approach to machine learning - collects and processes all of the data on a single site before making an detection. While this approach is effective at detecting attacks, there are a number of limitations associated with its use; for example, each time data is collected the privacy of that data is compromised. In addition, the amount of data collected creates very high levels of overhead, and there is added risk that the single point of failure will be compromised. Also, the current network environment generates very large - scale - or distributed - amounts of sensitive data, so there is no way to securely share this raw data or to safely transfer it between different regions/entities. This is exceedingly important to protect sensitive and secure data, and to protect networks from future breaches - particularly enterprises, cloud computing, and critical infrastructure. Another significant concern is the large number of different types of data that are produced from each client or source that generate a single aggregate output of data; these differences in client datasets cause degradation of performance of the centralized model. Additionally, the prevalence of class imbalance poses a challenge to accurately identifying rare - yet still critical - attack types, such as R2L and U2R. The need now is to implement a distributed, efficient/intrusion detection mechanism that adheres to the principles of privacy preservation through no longer sharing of sensitive raw data and handles distributed/heterogeneous sources of data.

1.3 LITERATURE SURVEY

There have been many advances in the research of intrusion detection, from simple signature-based systems to much more sophisticated paradigms based on machine learning (ML) and deep learning (DL). Tavallaee et al. [1] developed the NSL-KDD dataset to serve as an enhanced alternative to the KDD Cup 1999 dataset to better address the major concerns associated with redundancy and class imbalance found in the KDD Cup 1999 dataset. Since then, NSL-KDD has come to be widely used as the de facto standard for benchmarking intrusion detection systems (IDS), and the five-class categorization (Normal, DoS, Probe, R2L, U2R) they provided represents a realistic and challenging testing environment for evaluating IDSs.

i. Machine Learning-Based Methods for IDS

The initial applications of machine learning-based IDS were limited to the application of a number of different machine-learning techniques including support vector machines (SVM), random forests, k-nearest neighbors, and naive bayes. Debar et al. [8] laid down a foundation of taxonomic categories of intrusion-detection techniques. Through ensembling techniques, researchers have shown that ensemble-based IDSs will generally exceed 90% detection rates when applied to the KDD-derived datasets; however, shallow machine learning models cannot capture the underlying temporal dependencies nor hierarchical feature representations in sequential network traffic that are inherently required to detect intrusions. Kasongo [16] demonstrated through a systematic comparison that tree-based ensemble-based methods perform significantly better than traditional SVM-based methods when applied to high-dimensional space representation of network traffic features; however, these methods are also inferior to deep architectures when applied to the multi-class problem on NSL-KDD. Recently, Kasongo [16] developed a framework for IDS using comparisons of multiple RNN-based architectures -- LSTM, GRU, and SRNN -- and used XGBoost-based feature selection on NSL-KDD and UNSW-NB15 datasets; he confirmed that feature reduction is necessary for successful generalization when dealing with high dimensional network traffic data and that LSTM-based models produce superior multi-class detection performance compared to other forms of RNN-based models.

ii. Deep Learning for IDS

Sequential network flow data is suitable for Recurrent Neural Networks (RNNs); specifically, Long Short-Term Memory (LSTM)-based RNNs are highly effective at capturing long-term temporal relationships in sequential network flows. Yin et al. [4] demonstrated that LSTM-based IDSs achieve higher levels of accuracy than do traditional ML-based IDSs on NSL-KDD; they achieved 97.40% detection accuracy. Convolutional Neural Networks (CNNs) are able to treat flow feature vectors as one-dimensional signals that represent spatial hierarchies and allow CNNs to learn hierarchical features through stacked convolutional filters. Vinayakumar et al. [10] designed a deep CNN framework for detecting intrusions in network flows and achieved 96.90% accuracy on NSL-KDD. Many researchers have designed hybrid CNN-LSTM frameworks that combine the ability of CNNs to learn local features and the

ability of LSTMs to model temporal dependencies in sequential flows. Kim et al. [11] demonstrated that a hybrid CNN-LSTM model achieves 97.10% accuracy on NSL-KDD and performs significantly better than do single architecture baseline models. Hnamte and Hussain[17] expanded upon this concept and proposed a deeper hybrid architecture called DCNNBiLSTM that combines deep CNN with bidirectional LSTM. They achieved 98.27% multi-class classification accuracy on NSL-KDD, the best reported result for a centralized hybrid deep neural network architecture on this benchmark. Their results demonstrate that bidirectional modeling of temporal dependencies produces statistically significant improvements in performance over unidirectional LSTM modeling. Bukhari et al. [18] extended this approach to apply it to wireless sensor networks by developing a federated SCNN-BiLSTM framework. In addition to demonstrating that federated architectures can successfully implement secure and privacy-preserving intrusion detection without requiring the centralization of sensitive network-flow data, Bukhari et al., also demonstrated that their federated architecture could function effectively in environments where there may be malicious participants attempting to disrupt normal operations. Deep Neural Networks (DNNs) with multiple fully connected layers can achieve competitive performance by performing hierarchical feature abstraction; specifically, Khan's HCRNNIDS achieved 97.80% accuracy on NSL-KDD [9].

iii. Federated Learning for Network Security

Federated Averaging (FedAvg) was first introduced by McMahan et al. [2]; FedAvg enables distributed training of a global model while maintaining low communication requirements through efficient gradient aggregation. FedProx was later introduced by Li et al. [3]; FedProx extends FedAvg with proximal regularization to constrain parameter divergence between client models and the global model -- crucial for addressing statistical heterogeneity in real-world network traffic. Nguyen et al. [6] conducted a survey of Federated Learning (FL) in cybersecurity and identified IDS as a high priority use case due to the sensitivity of network-flow data and strict regulatory requirements. Mothukuri et al. [12] conducted a review of privacy preserving FL for IoT security and demonstrated that sharing gradients can expose sensitive information about clients through model inversion attacks. However, Rey et al. [13] demonstrated the feasibility of designing a federated IDS for IoT devices that achieved high levels of detection while exhibiting FL robustness to Byzantine failures. Additionally, Al-Marri et al. achieved 98.11% detection accuracy on NSL-KDD using federated mimic learning with MLP backbones. Furthermore, Zhao et al. [14] investigated the non IID problem in FL and demonstrated that even small amounts of globally shared data can greatly improve convergence speed. Finally, Friha et al.'s [19] recent comprehensive review of FL-IDS research contributed during 2017 – 2024 systematically analyzed deep learning based FL-IDS contributions in both network and IoT domains; non IID data handling, communication overhead, and mitigating untrusted participants were identified as the three primary open challenges -- all of which motivated the architecture level comparative study presented here. Most recently, Deshmukh et al. [20] demonstrated that standard FL algorithms can obtain similar results to centralized approaches while dramatically reducing communication overhead; this result supports and validates the empirical evidence presented in section four of this paper.

1.4 ARCHITECTURE DIAGRAMS

Two parallel experimental pathways are evaluated: First, a centralized framework will be tested where all data will reside on one server. Second, a federated learning framework will be tested where all training data is horizontally partitioned across 5 simulated clients.

we have evaluated five different deep learning models based on their architectures and capabilities to classify the data. These include Multilayer Perceptron (MLP), Long Short Term Memory (LSTM), Convolutional Neural Networks (CNN), Deep Neural Networks (DNN) and Hybrid CNN + LSTM.

1.5 HARDWARE AND SOFTWARE REQUIREMENTS

The Hardware requirements are

1. Processor : Intel Core i5 or above
2. RAM : 8 GB minimum
3. Hard Disk : 256 GB or above
4. System Type : 64-bit Operating System
5. Internet Connectivity

And the Software requirements are

1. Operating System : Windows/Linux/macOS
2. Front End : Python (Google Colab)
3. Back End : TensorFlow, Keras
4. Libraries : NumPy, Pandas, Scikit-learn, Matplotlib
5. Dataset : NSL-KDD
6. Algorithm : Federated Learning (FedProx), Deep Learning Model

1.6 MODULES AND DESCRIPTION

1.6.1 DATASET DESCRIPTION

The NSL-KDD dataset is made up of 148,520 labeled samples with four classes of attack: DoS (Denial of Service), Probe, R2L (Remote to Local), and U2R (User to Root). In addition, there is a fifth class called "Normal." There are 29,704 samples in the test set and 118,816 samples in the training set. I have used all 41 features of the dataset. All categorical variables were one hot coded and normalized to [0, 1]. I had a very high imbalance ratio with the Normal variable being ~52%, DoS ~36%, Probe ~10%, R2L ~1.5% and U2R ~0.3%. Because of this imbalance ratio, there are many challenges with detecting minority classes.

1.6.2 PREPROCESSING

DEEP LEARNING - DESCRIPTION

Data Collection

Data from different sources is collected to generate network traffic.

In this case, NSL-KDD dataset is used for training and testing purposes.

The data contains both types of traffic, i.e., normal and attack traffic.

Data Integration

All the collected data is then integrated and stored in one system.

This enables all data to be processed in one place.

Moreover, it becomes easy to train the model using complete data set.

Feature – Label Separation

The data set is split into two categories; i.e., feature and label.

Feature represents input data values and describes the characteristics of data.

The label denotes whether the traffic is normal or falls into any of the attack categories.

Attack Category Mapping

Attack traffic is mapped under five major categories, including DoS, Probe, R2L, U2R, and Normal.

It makes classification easier and more systematic.

One-Hot Encoding

One-Hot encoding technique is used to convert categorical values to numerical values.

Binary values are used for each category.

Feature Scaling

Features are scaled such that they fall in a common range.

This helps in reducing the bias created by large values affecting small values.

This improves the performance of the model.

Feature Selection

Relevant features that help in prediction are selected.

Unnecessary features, which are redundant, are omitted.

This results in accuracy and efficiency of the algorithm.

Model Training

All data is utilized in training the model using centralization.

Models like MLP, DNN, CNN, LSTM, and CNN+LSTM are trained.

Training using all the data makes the model capture the global patterns.

Model Evaluation

Scores like accuracy, precision, recall, and F1 are utilized in the evaluation process.

Confusion Matrix is another method used to evaluate model performance.

Prediction

Models that are trained are used to predict classes of test data.

FEDERATED LEARNING – DESCRIPTION

Initialization of clients

Many clients exist during the training process.

Every client has its own datasets which remain unique.

They are exclusive for use within that particular client only.

Global model initialization

The central server initializes a global model.

After that, parameters of that model are sent to all clients.

This model will be used to kickstart the training process.

Local training process

The clients train the model using their individual dataset.

Training is done using different neural networks, e.g., MLP, DNN, CNN, and LSTM.

Learning happens using the client's local dataset.

Model parameter sharing

During the training process, the client does not share its local data.

But instead, the server shares the updated models.

Data security and confidentiality can be maintained.

Model aggregation using fedprox

After completing the local training process, the models are aggregated.

Aggregation process is done using FedProx.

Global model update

The global model is created after combining the model updates by the server.

Again, the updated global model is distributed to all clients.

The above cycle is continued for several rounds.

Training iteration

This continues until a better performing model is obtained.

At each iteration, the model is improved in terms of its accuracy.

Overall, the learning gets improved for all clients.

Evaluation of global model

The model that comes at the end is evaluated using validation/test data.

The performance is calculated using Accuracy, Precision, Recall, and F1 score.

Privacy protection

The original data is always retained on the client side.

Updates to models are shared in the training process.

Thus, privacy is guaranteed by the process

1.7 RESULTS AND DISCUSSION

In this section we will show results of experiments with all 5 deep architectures in both centralized and federated training modes that were tested using the holdout testing set of 29,704 examples from the NSL-KDD dataset. We will follow up with a class-by-class breakdown of each model's performance and compare it to what has been reported in prior literature as the current "state of the art".

Centralized Learning Results

Classification performance of the five architectures are presented. In terms of centralized accuracy and F1-score the best performing architecture is the DNN with accuracy = 97.14% and F1-score = 97.46%. The CNN and LSTM architectures have similar performance with an accuracy of 96.73% and 96.72%, respectively. The worst performance was observed for the hybrid CNN + LSTM architecture which had a lower accuracy at 95.32% than either CNN or LSTM due to having a higher number of parameters that need to be trained from a single data source to reach convergence.

Centralised Deep Learning Model Performance on NSL-KDD

Federated Learning Results

This table shows the federated training results by using the Federated Proximal (FedProx) algorithm. Amongst the many different models that trained with Federated Lstm; Federated Lstm was the most accurate with a 97.55% accuracy, while Federated Mlp had a slightly lower accuracy of 97.49%. The remaining models demonstrated significantly lower accuracies than the top two models. These include: Dnn (accuracy = 97.16%), Cnn (accuracy = 97.0%) and Cnn+Lstm (accuracy= 96.42%). In addition, all of the federated models outperformed their centralized counterpart on each of the four metrics.

Federated Learning Model Performance on NSL-KDD (FedProx)



CHAPTER 2

MERITS AND DEMERITS OF BASEPAPER

2.1 MERITS

Enhances Privacy of User Data

When users leverage federated learning, they can train the model and never share their private data with others. Therefore, a user who uses federated learning is at much less risk of having their information compromised by other clients/parties.

Lower Centralized Dependency

Unlike a traditional centralized IDS architecture that relies on having one central server in the system, federated learning allows a number of different decentralized clients to have their own models without having a single, central point of failure.

Higher Model Generalization

Federated learning allows training the model on many different clients (and their corresponding datasets), thus enabling it to learn from multiple data sources and achieve better generalization of the overall model and accuracy of the detection of all entities in the model.

Better The Use of Heterogeneous Data

Using a FedProx federated learning algorithm can assist in handling non-identical data that are present among the multiple clients of a federated learning system.

Scalability of the Architecture

Because adding additional or new clients is an easy process in a federated learning architecture, scaling the architecture will not require significant changes to the federated learning architecture.

Reduced Communication Load Compared to Transmission of Raw Data

The federated learning architecture only requires communication with clients via the transmission of model update requests rather than transmitting all of the original datasets to the client, thus allowing for increased conservation of bandwidth.

2.2 DEMERITS

High Computational Cost for Clients

The process of training local models by each client can consume a lot of resources on devices with less processing power.

Communications Still Create Latency

Although they are reduced, the amount of time it takes for clients to regularly communicate with the server can still introduce some latency.

Class Imbalance



Many intrusion systems will have many more samples for attacks than others (e.g., R2L and U2R), resulting in a lower detection rate for these classes.

More Complex to Design, Implement, and Debug

Federated learning systems are harder to design, implement, and debug than centralized learning systems.

Convergence Issues

Because the data being used is not IID, it may take a long period of time for the global models to converge and there will likely be fluctuations in the model during training.

Security Issues with Model Updates

Even though the actual data is not being shared, malicious clients could still submit poison updates that would affect the global model.

CHAPTER 4

Analysis of Confusion Matrices:

Confusion matrices help to evaluate how effective the developed intrusion detection models have been created to identify each of the classes based on **comparison of the predicted classes and the actual classes**. The entries on the diagonal of the confusion matrices show the instances that have been **correctly predicted (i.e., true positives and true negatives)** and the entries that are not located on the diagonal of the confusion matrices represent instances that have been **incorrectly predicted (i.e., false positives and false negatives)**. The majority of instances located on the diagonal of the confusion matrices indicate a high accuracy of the models in predicting the intrusions. The most common classes (Normal, Dos, and Probe) have all been correctly identified with very few errors in their classifications. However, there are some errors in classifying the incorrect class labels of the **minority classes (R2L and U2R)** resulting from **the class imbalance in the NSL-KDD dataset**. The results of the comparison between the centralized and federated confusion matrices indicate that the federated model has lower error rates than the centralized model. In the above figure, the test accuracies of deep learning (DL) central models using deep learning (DL) based on the FedProx aggregation method (federated learning) is depicted. It can be seen that, regardless of the model, federated models, on average, provided at least equal or superior performance relative to central models.

The best model overall, the Federated LSTM, achieved a test accuracy of 97.55%; the second best performance was recorded by the Federated MLP (97.49%) indicating they are superior intrusion detection capabilities. There was a small improvement to the DNN, while there was a noticeable improvement to the **CNN (Convolutional Neural Network), LSTM e.g., Long Short-Term Memory, MLP, and CNN with LSTM** under federated training.

Ultimately, the **results demonstrate that federated learning** not only **allows** preservation of privacy, but it also increases classification performance when compared to DL-based, conventional central learning solutions.

4.1 PERFORMANCE ENHANCEMENT ANALYSIS OF FL OVER DL

The presented graphs indicate the accuracy improvements made by Federated Learning when compared to Centralized Learning on each individual model. Each model has produced overall positive accuracy improvements and indicates that the Federated Learning approach that was presented works.

The greatest amount of improvement for all models was seen with the Multi Layer Perception (MLP) at 0.0157, then continuing to Convolutional Neural Network with Long Short Term Memory (CNN + LSTM) with an accuracy improvement of 0.0110, and Long Short Term Memory (LSTM) with an accuracy improvement of 0.0083; thus, providing strong evidence of the added performance benefit of using Federated Learning compared to using Centralized Learning. CNN had a moderate amount of improvement while the Deep Neural Networks (DNN) showed only a very small improvement; however, DNN's had very accurate performance when using Centralized Learning.

These improvements provide strong evidence that the FedProx based Federated Learning framework provides an increase in model generalization and an increase in intrusion detection accuracy while still providing a method for privacy preserved, distributed learning.

4.2 CONVERGENCE ANALYSIS

The federation's performance over time is illustrated in the convergence graph by the number of rounds of communication that have happened. In the first few rounds of communication, performance increased significantly (increased dramatically), whereas performance no longer improves very much (remains fairly unchanged) for the remainder of the communication rounds. The global model appears as if it is converging based on the upward trend of the accuracy or downward trend in the loss function. Once convergence occurs (reaches stabilization), other than having a very small degree of variability, indicates that using FedProx has provided a consistent and effective method to learn from distributed datasets. The results of this research provide evidence demonstrating that the proposed system is capable of learning effectively regarding both robustness and the ability to generalize well.

CHAPTER 5

CONCLUSION AND FUTURE PLANS

5.1 CONCLUSION

In this study we examined a complete comparative evaluation of the effectiveness of five deep learning architectures -- MLP, LSTM, CNN, DNN, and Hybrid CNN+LSTM for network intrusion detection using the NSL-KDD Benchmark Dataset. In addition, we also evaluated each of these architectures using two different machine learning paradigms (centralized and federated) for evaluating the architectures. We found four major points from our experiments. First, federated learning provides similar or better results than centralized learning when using either accuracy, precision, F1 score, and recall as metrics. In particular, we were able to show that Federated LSTMs achieved an overall accuracy of 97.55% which exceeded the best centrally trained model (the DNN at 97.14%). At the same time, we ensured that none of the raw network traffic records left any one of the individual nodes.

Second, the FedProx aggregation method has been very successful in stabilizing the federated convergence process even though there is a large disparity in the distribution of the data. Consequently, we have shown that all five architectures can achieve some form of stable, plateau-like performance by round 20. Additionally, we found that all of the federated models surpassed their respective centrally-trained baselines after 3-5 rounds. These findings confirm both the superior performance and communication efficiency of federated learning versus centralized learning.

Third, in per-class analysis, we found that DoS and Probe attacks are consistently identified ($F1 > 96.7\%$), however, we still found significant challenges with detecting R2L and U2R minority classes in both paradigms. Specifically, we believe that class imbalance will continue to be a field-wide limitation for IDSs in the future -- therefore future work should focus on addressing this issue through augmentations and/or cost-sensitive learning strategies.

Fourth, comparing our proposed federated-learning based approaches with six previously-published state-of-the-art approaches demonstrated that the proposed federated-learning approaches are competitive with the current research frontier. Furthermore, our proposed multi-architecture comparative framework provides additional scientific value beyond just demonstrating competitiveness. Our framework demonstrates how a practitioner could select architectures for use in an FL-IDS environment based on evidence obtained from empirical evaluations of multiple architectures. Therefore, the primary contributions of our work is providing a comparative evaluation of architectures in a federated paradigm, providing evidence-based selections of architectures for use in FL-IDS environments, and identifying areas where future work needs to occur.

5.2 FUTURE WORK:

There are several potential avenues for future research based upon our findings. Some possible avenues for future work include: (i) Evaluating our frameworks using Dirichlet distributions to simulate realistic organizational traffic heterogeneity rather than IID data partitions; (ii) Incorporating differential privacy into our frameworks using Gaussian noise added to gradient values in order to provide formal privacy guarantees against model inversion attacks; (iii) Extending our frameworks to utilize three new benchmarks (CICIDS-2017, UNSW-NB15, and CIC-IoT-2023) so that we may assess generalization across different datasets; (iv) Applying

SMOTE oversampling techniques along with cost sensitive loss functions and class weighted FedProx aggregation to improve detection rates for R2L and U2R minority classes; and (v) Developing asynchronous federated learning protocols for clients with varying computational capabilities and/or those whose networks experience periods of disconnection.

CHAPTER 6

APPENDIX

7.1 Dataset Description

The NSL-KDD dataset is used for training and testing the intrusion detection systems.

- Total number of samples: 148,520
- Number of features: 41
- Classes: Normal, DoS, Probe, R2L, U2R

The dataset is imbalanced, with only a small number of instances in the R2L and U2R classes.

7.2 Preprocessing Procedure

The preprocessing steps include:

- Removal of invalid entries and duplicate records
- Conversion of categorical features using One-Hot Encoding
- Application of feature scaling
- Selection of important features using Mutual Information
- Handling class imbalance using class weights

7.3 Model Setup

Centralized Learning:

- Optimizer: Adam
- Number of epochs: 10
- Batch size:

128 Federated

Learning:

- Number of clients: 5
- Number of rounds: 20
- Algorithm used: FedProx

7.4 Evaluation Criteria

Model performance is evaluated using the following metrics:

- Accuracy
- Precision
- Recall
- F1 Score
- Confusion Matrix

7.5 Important Findings

- Federated learning performs better than centralized learning
- Data privacy is maintained as raw data is not shared
- Minority classes such as R2L and U2R are more difficult to classify

6.6 Software Environment

The project is implemented using the following tools and libraries:
Python, TensorFlow, Keras, NumPy, Pandas, Scikit-learn, and Matplotlib

6.7 Conclusions

This appendix provides additional details about the dataset, preprocessing steps, model configuration, and evaluation methods used in the project.